

Hanoi University of Science and Technology
School of Information and Communications Technology



Diffusion Models

Tran Nam Hai

February 20, 2026

Contents

1	Overview	3
1.1	The Rise of Generative AI	3
1.2	Evolution of Generative Models	3
1.3	Problem Statement	3
2	Theoretical Foundation	4
2.1	Denoising Diffusion Probabilistic Models	4
2.1.1	Forward Diffusion Process	4
2.1.2	Reverse Diffusion Process	5
2.2	Noise-Conditioned Score Network	8
2.3	Denoising Diffusion Implicit Models	9
2.4	Rectified Flow and Flow Matching	10
2.4.1	Velocity Prediction	10
2.4.2	Rectified Flow	11
2.4.3	Flow Matching	13
2.5	Rectified Diffusion	13
2.5.1	InstaFlow	14
2.5.2	PeRFlow	14
3	Model Architectures	15
3.1	Denoising Backbone	15
3.1.1	U-Net	15
3.1.2	Diffusion Transformer	16
3.2	Conditioning Mechanisms	17
3.2.1	Text Encoders	18
3.2.2	Timestep Embeddings	19
3.2.3	Adaptive Normalization	20
3.2.4	Cross Attention	21
3.3	Perceptual Compression	22
3.4	Guidance Methods	23
3.4.1	Classifier Guidance	23
3.4.2	Classifier-Free Guidance	24
3.4.3	Guidance Distillation	24
4	Adaptation Modules	25
4.1	Weight Injection Adapters	25
4.1.1	Low-Rank Adaptation	25
4.1.2	HyperNetworks	27
4.1.3	HyperAlign	27
4.2	Feature Injection Adapters	28
4.2.1	ControlNet	28
4.2.2	BrushNet	29
4.3	Attention Injection Adapters	30
4.3.1	IP-Adapter	30
5	Open Source Models	31
5.1	Stable Diffusion	32
5.1.1	Stable Diffusion 1.x	32
5.1.2	Stable Diffusion 2.x	32
5.1.3	Stable Diffusion XL	33
5.1.4	Stable Diffusion 3.x	33
5.2	FLUX	34
5.3	Hunyuan-DiT	35
5.4	Kandinsky	36

1 Overview

1.1 The Rise of Generative AI

The landscape of Artificial Intelligence has undergone a shift in recent years, moving from discriminative models (designed to classify and analyze existing data) to generative models capable of synthesizing entirely new data samples. This paradigm shift, often termed the “Generative AI Revolution”, has fundamentally altered how we approach digital content creation, particularly in the domain of computer vision.

Historically, computer vision focused on understanding the “what” and “where” within an image. Today, Generative AI allows us to answer “what if?”. By learning the underlying probability distribution of vast datasets, modern models can generate high-fidelity images that are indistinguishable from reality. The democratization of these tools, through interfaces like Midjourney [1], DALL-E [2], and Stable Diffusion [3], has empowered users to create complex visual assets using nothing but natural language. However, the true scientific marvel lies not just in the visual output, but in the sophisticated mathematical frameworks that enable computers to “dream” and, more importantly, to be precisely guided by human intent.

1.2 Evolution of Generative Models

To understand the current state-of-the-art (SOTA) in text-to-image (T2I) generation, it is essential to trace the evolutionary lineage of these models. The journey has been characterized by a constant trade-off between sample quality, diversity, and computational efficiency.

The early era of deep generative models was marked by **Variational Autoencoders (VAEs)** [4]. By compressing data into a lower-dimensional latent space and maximizing the evidence lower bound (ELBO), VAEs provided a probabilistic framework for generation. However, while mathematically elegant, VAEs often suffered from the “blurriness” problem, failing to capture high-frequency details effectively.

Proposed by Goodfellow et al., **Generative Adversarial Networks (GANs)** [5] introduced a game-theoretic approach in which a Generator competes against a Discriminator. GANs solved the blurriness issue, producing razor-sharp images that set new benchmarks for realism. Despite their success, GANs are notoriously difficult to train, suffering from non-convergence and “mode collapse” (where the model generates a limited variety of samples), making them less ideal for large-scale, diverse T2I tasks.

The introduction of **Diffusion Models** [6] marked a turning point. Inspired by non-equilibrium thermodynamics, these models learn to reverse a gradual noising process. Unlike GANs, diffusion models offer stable training and cover the data distribution (high diversity). By iteratively denoising a random Gaussian distribution, models like Stable Diffusion have become the standard for T2I generation.

The most recent evolution is the shift toward **Rectified Flow** [7] and **Flow Matching** [8]. While standard diffusion models follow stochastic differential equations (SDEs) often resulting in curved sampling trajectories, Flow Matching aims to straighten these paths. By learning an Ordinary Differential Equation (ODE) that connects the noise distribution to the data distribution via the straightest possible path, these models achieve higher efficiency and superior prompt adherence, paving the way for more precise image synthesis.

1.3 Problem Statement

While the foundational capability to synthesize high-fidelity images from pure noise has reached remarkable maturity, the challenge of **Precise and Controllable Text-to-Image Generation** remains a significant open problem.

The core conflict in modern generative modeling lies in the tension between *expressive diversity* and *strict conditional control*. When a user provides a complex, multi-clausal prompt or a specific spatial constraint, the ideal outcome requires:

- **Semantic Alignment (Prompt Adherence):** The model must accurately render all subjects, attributes, and spatial relationships specified in the text without omitting details or suffering from “attribute bleeding” (e.g., mixing up the colors of different objects).
- **Aesthetic and Human Preference Alignment:** The generated output should not only match the prompt but also exhibit high aesthetic quality and structural coherence that aligns with human visual expectations, without requiring endless “prompt engineering”.

- **Controllability vs. Efficiency:** Achieving precise spatial or stylistic control often requires either computationally expensive full-model fine-tuning, which risks catastrophic forgetting and loss of diversity, or slow test-time optimizations.

Standard unconstrained diffusion generation is inherently stochastic; ensuring that a model reliably converges on an image that perfectly matches a highly specific human intent is mathematically difficult.

This document focuses on the cutting-edge solutions to these generative dilemmas. By exploring advanced conditioning mechanisms, parameter-efficient adaptation modules, and SOTA foundational architectures, we analyze how modern models mathematically guarantee a higher degree of prompt adherence and structural control, offering a robust framework for the future of controllable image synthesis.

2 Theoretical Foundation

Diffusion Models were first proposed in 2015 based on the concept of using a diffusion process to gradually destroy the data structure and subsequently build a generative model learned to reverse this process. By 2020, two independent approaches emerged that improved upon this concept and established the paradigm for modern diffusion models:

- **Noise-Conditioned Score Network** (Yang & Ermon, 2019) [9]
- **Denoising Diffusion Probabilistic Models** (Ho et al. 2020) [10]

Of these two approaches, Denoising Diffusion Probabilistic Models are the more prominent and accessible framework. However, Noise-Conditioned Score Networks also play a critical role in the theoretical foundation of modern diffusion models.

Subsequently, the field witnessed a paradigm shift around 2022, moving from standard noise prediction to velocity prediction. This geometric perspective laid the groundwork for Rectified Flow and Flow Matching (2022).

2.1 Denoising Diffusion Probabilistic Models

Fundamentally, a **Denoising Diffusion Probabilistic Models** (DDPM) [10] is a neural network that learns to iteratively denoise data, starting from pure noise. The model consists of two processes:

- **Forward Diffusion Process** (Fixed): Gradually adds Gaussian noise to the data.
- **Backward Diffusion Process** (Learned): The network is trained to gradually remove the noise.

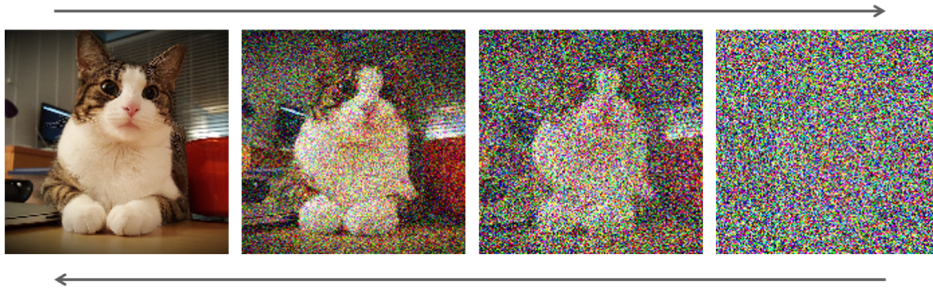


Figure 1: Forward and Backward Diffusion Process

2.1.1 Forward Diffusion Process

The Forward Process is a fixed chain of probabilistic transformations aimed at converting real data x_0 into pure noise x_T . We define the transition probability from step $t - 1$ to t as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t)$$

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

At each step, the original data signal is slightly attenuated, and the noise is slightly increased. Here, β_t is referred to as the **Variance Schedule**, which determines the rate of data corruption.

However, iteratively sampling x_t step-by-step ($x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_t$) is computationally expensive and slow for training. By leveraging the **Reparameterization Trick** and the property that the sum of two independent Gaussian distributions is also a Gaussian ($\mathcal{N}(\mu_1, \sigma_1^2) + \mathcal{N}(\mu_2, \sigma_2^2) \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$), we can merge all intermediate steps into a single closed-form alignment:

$$\begin{aligned}
x_t &= \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_{t-1} \\
&= \sqrt{1 - \beta_t} \left(\sqrt{1 - \beta_{t-1}}x_{t-2} + \sqrt{\beta_{t-1}}\epsilon_{t-2} \right) + \sqrt{\beta_t}\epsilon_{t-1} \\
&= \sqrt{(1 - \beta_t)}\sqrt{(1 - \beta_{t-1})}x_{t-2} + \sqrt{(1 - \beta_t)}\sqrt{\beta_{t-1}}\epsilon_{t-2} + \sqrt{\beta_t}\epsilon_{t-1} \\
&= \sqrt{(1 - \beta_t)}\sqrt{(1 - \beta_{t-1})}x_{t-2} + \sqrt{(1 - \beta_t)\beta_{t-1} + \beta_t}\bar{\epsilon} \\
&= \sqrt{(1 - \beta_t)(1 - \beta_{t-1})}x_{t-2} + \sqrt{1 - (1 - \beta_t)(1 - \beta_{t-1})}\bar{\epsilon} \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon} \\
&\dots \\
\Rightarrow x_t &= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\bar{\epsilon} \Rightarrow q(x_t|x_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}x_0, 1 - \bar{\alpha}_t)
\end{aligned}$$

In this derivation, we utilize the property of adding Gaussian noise to jump directly from x_0 to x_t . Here, $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ represents the cumulative product of the variance schedule terms.

2.1.2 Reverse Diffusion Process

If we could reverse the forward process and sample from $q(x_{t-1}|x_t)$, we would be able to recreate the true sample from a Gaussian noise input $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. However, estimating $q(x_{t-1}|x_t)$ directly is intractable because it requires knowledge of the entire dataset distribution. Therefore, we must learn a model p_θ to approximate these conditional probabilities.

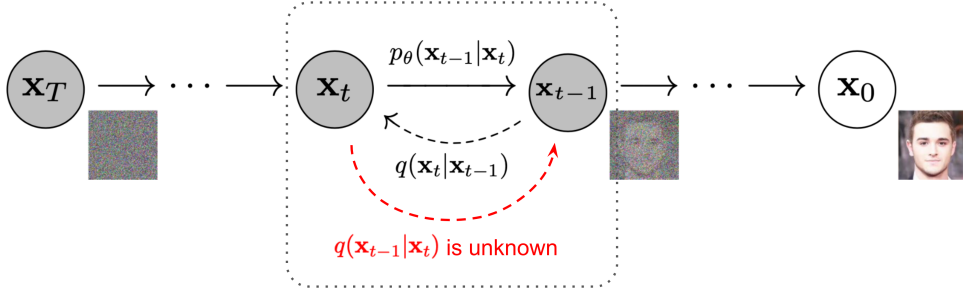


Figure 2: Backward Diffusion Process

A crucial theoretical foundation for Diffusion Models is the behavior of the reverse step when the step size is small. If the variance schedule β_t is sufficiently small, the reverse transition $q(x_{t-1}|x_t)$ can be approximated as a Gaussian distribution. From Bayes' theorem, the true reverse posterior is given by:

$$q(x_{t-1}|x_t) = \frac{q(x_t|x_{t-1})q(x_{t-1})}{q(x_t)} \propto q(x_t|x_{t-1})q(x_{t-1})$$

Recall that the forward transition is defined as Gaussian:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t) \propto \exp\left(-\frac{\|x_t - \sqrt{1 - \beta_t}x_{t-1}\|^2}{2\beta_t}\right)$$

When β_t is infinitesimal, the variance β_t in the denominator becomes extremely small, causing the likelihood term $q(x_t|x_{t-1})$ to become highly peaked (very narrow). In contrast, the data distribution term $q(x_{t-1})$ varies much more smoothly locally. According to Sohl-Dickstein et al. [6] and Feller's theory on stochastic processes, the product of a smooth prior $q(x_{t-1})$ and a highly concentrated Gaussian likelihood $q(x_t|x_{t-1})$ is dominated by the Gaussian form of the likelihood. Consequently, the log-posterior can be

approximated by a quadratic function with respect to x_{t-1} , which implies that the distribution itself is Gaussian:

$$q(x_{t-1}|x_t) \approx \mathcal{N}(x_{t-1}; \boldsymbol{\mu}_q(x_t, t), \boldsymbol{\sigma}_q(x_t, t))$$

This theoretical guarantee allows us to parameterize our neural network p_θ simply as a Gaussian predictor:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \boldsymbol{\mu}_\theta(x_t, t), \boldsymbol{\sigma}_\theta(x_t, t))$$

While it is theoretically possible to learn both the mean $\boldsymbol{\mu}_\theta$ and the variance $\boldsymbol{\sigma}_\theta$, empirical studies suggest that learning both simultaneously is unstable and difficult to converge. Consequently, a common design choice is to fix the variance to a time-dependent constant and focus the network solely on learning the mean $\boldsymbol{\mu}_\theta$.

Although $q(x_{t-1}|x_t)$ is intractable, the posterior conditioned on the original data x_0 , denoted as $q(x_{t-1}|x_t, x_0)$, is tractable. Using Bayes' rule:

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= q(x_t|x_{t-1}, x_0) \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} \\ &= \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, \beta_t) \frac{\mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, 1 - \bar{\alpha}_{t-1})}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, 1 - \bar{\alpha}_t)} \\ &\propto \exp\left(-\frac{1}{2}\left(\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(x_t - \sqrt{\bar{\alpha}_t}x_0)^2}{1 - \bar{\alpha}_t}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\frac{x_t^2 - 2\sqrt{\alpha_t}x_tx_{t-1} + \alpha_tx_{t-1}^2}{\beta_t} + \frac{x_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}x_0x_{t-1} + \bar{\alpha}_{t-1}x_0^2}{1 - \bar{\alpha}_{t-1}} - C(x_t, x_0)\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right)x_{t-1}^2 - 2\left(\frac{\sqrt{\alpha_t}}{\beta_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}x_0\right)x_{t-1} + C(x_t, x_0)\right)\right) \end{aligned}$$

where $C(x_t, x_0)$ represents terms not involving x_{t-1} . Following the standard Gaussian density function definition $\mathcal{N}(x; \mu, \sigma^2) \propto \exp(-\frac{1}{2\sigma^2}(x - \mu)^2)$, we can identify the variance $\tilde{\beta}_t$ and mean $\tilde{\boldsymbol{\mu}}_t(x_t, x_0)$ by matching coefficients:

$$\begin{aligned} \tilde{\beta}_t &= 1 / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right) = 1 / \left(\frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})}\right) = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t \\ \tilde{\boldsymbol{\mu}}_t(x_t, x_0) &= \left(\frac{\sqrt{\alpha_t}}{\beta_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}x_0\right) / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right) \\ &= \left(\frac{\sqrt{\alpha_t}}{\beta_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}x_0\right) \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t \\ &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 \end{aligned}$$

Since we do not know x_0 during inference, we use the reparameterization property of the forward process, $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$, to express x_0 in terms of x_t :

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon})$$

Substituting this expression for x_0 back into the above equation for $\tilde{\boldsymbol{\mu}}_t$, we obtain the formulation for the mean that the network must predict:

$$\begin{aligned} \boldsymbol{\mu}_\theta(x_t, t) &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}\right) \\ &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(x_t, t)\right) \end{aligned}$$

The training objective is to minimize the Kullback-Leibler (KL) divergence [11] between the true posterior $q(x_{t-1}|x_t, x_0)$ and the modeled distribution $p_\theta(x_{t-1}|x_t)$.

Since we determined earlier that both distributions are Gaussian:

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= \mathcal{N}(x_{t-1}; \tilde{\boldsymbol{\mu}}_t(x_t, x_0), \tilde{\beta}_t) \\ p_\theta(x_{t-1}|x_t) &= \mathcal{N}(x_{t-1}; \boldsymbol{\mu}_\theta(x_t, t), \boldsymbol{\sigma}_\theta(x_t, t)) \end{aligned}$$

The KL divergence between two multivariate Gaussian distributions $\mathcal{N}(\mu_1, \sigma_1)$ and $\mathcal{N}(\mu_2, \sigma_2)$ is dominated by the quadratic difference between their means. Specifically:

$$D_{KL}(\mathcal{N}(\mu_1, \sigma_1) || \mathcal{N}(\mu_2, \sigma_2)) \approx \frac{1}{2\sigma_2^2} \|\mu_1 - \mu_2\|^2 + C$$

Thus, minimizing the loss L_t is equivalent to minimizing the difference between the true mean $\tilde{\boldsymbol{\mu}}_t$ and the predicted mean $\boldsymbol{\mu}_\theta$:

$$\begin{aligned} L_t &= D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t)) \\ &= \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\|\boldsymbol{\sigma}_\theta(x_t, t)\|_2^2} \|\tilde{\boldsymbol{\mu}}_t(x_t, x_0) - \boldsymbol{\mu}_\theta(x_t, t)\|^2 \right] \end{aligned}$$

Recall our derived formula for the true posterior mean:

$$\tilde{\boldsymbol{\mu}}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} \right)$$

To match this structure, we parameterize the neural network $\boldsymbol{\mu}_\theta$ to predict $\boldsymbol{\epsilon}_\theta$ given x_t :

$$\boldsymbol{\mu}_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(x_t, t) \right)$$

Now, we substitute these mean formulations into the loss equation:

$$\begin{aligned} \|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta\|^2 &= \left\| \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} \right) - \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta \right) \right\|^2 \\ &= \left\| \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1}{\sqrt{\alpha_t}} x_t - \frac{\beta_t}{\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} + \frac{\beta_t}{\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta \right\|^2 \\ &= \left\| \frac{\beta_t}{\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t}} (\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(x_t, t)) \right\|^2 \\ &= \frac{\beta_t^2}{\alpha_t (1 - \bar{\alpha}_t)} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(x_t, t)\|^2 \end{aligned}$$

Substituting this back into the expectation L_t , and recalling that $\beta_t = 1 - \alpha_t$:

$$L_t = \mathbb{E}_{x_0, \epsilon} \left[\frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t)\|\boldsymbol{\sigma}_\theta\|_2^2} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(x_t, t)\|^2 \right]$$

In the standard DDPM implementation, the variance $\boldsymbol{\sigma}_\theta$ is fixed to a constant (either β_t or $\tilde{\beta}_t$) and is not learned. Additionally, α_t and β_t are predefined hyperparameters. Consequently, the term in front of the squared norm is merely a time-dependent weighting factor:

$$\lambda_t = \frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t)\|\boldsymbol{\sigma}_\theta\|_2^2}$$

Ho et al. (2020) empirically found that the model performs better if we simply discard this weighting term ($\lambda_t = 1$). This re-weights the loss to emphasize difficult denoising steps (typically at small t) and leads to the **Simplified Objective**:

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}, t)\|^2]$$

This reduces the complex KLDiv to a straightforward Mean Squared Error (MSE) loss between the actual noise $\boldsymbol{\epsilon}$ added to the image and the noise $\boldsymbol{\epsilon}_\theta$ predicted by the network.

Algorithm 1 Training	Algorithm 2 Sampling
1: repeat 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 5: Take gradient descent step on $\nabla_{\theta} \ \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\ ^2$ 6: until converged	1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 2: for $t = T, \dots, 1$ do 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{1 - \bar{\alpha}_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 5: end for 6: return $\mathbf{x}_0$

Figure 3: Training and Sampling Algorithms in DDPM

2.2 Noise-Conditioned Score Network

While DDPM focuses on learning a reverse Markov chain to denoise data, **Noise-Conditioned Score Networks** (NCSN) [9] approach generative modeling from a different perspective: **Score Matching**.

The core idea is that while learning the full probability density function $p(x)$ of real data is often intractable, learning its gradient with respect to the data, known as the **Stein Score Function**, is feasible. The score is defined as:

$$\mathbf{s}(x) = \nabla_x \log p(x)$$

A neural network, called the Score Network $\mathbf{s}_{\theta}(x)$, is trained to estimate this gradient field. Once the score function is learned, new data samples can be generated using an iterative process that moves random noise points along the gradient toward high-density regions of the data distribution:

$$x_{i+1} = x_i + \frac{\epsilon}{2} \nabla_x \log p(x_i) + \sqrt{\epsilon} \mathbf{z}_i, \quad \mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

A significant challenge in score matching is the **Manifold Hypothesis**: real-world high-dimensional data tends to concentrate on low-dimensional manifolds. This means that for the vast majority of the ambient space, the data density $p(x)$ is near zero, making the score estimation $\nabla_x \log p(x)$ undefined or unstable in those empty regions. To address this, NCSN perturbs the data with various levels of Gaussian noise. This “bloats” the data manifold to cover the entire space, ensuring the score is well-defined everywhere. The network $\mathbf{s}_{\theta}(x, \sigma)$ is then trained to jointly estimate the scores for all noise levels σ .

Although derived from different theoretical frameworks, NCSN and DDPM are **mathematically unified**. Recall from the DDPM forward process that $q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$. The logarithm of this Gaussian density is:

$$\log q(x_t|x_0) = -\frac{1}{2(1 - \bar{\alpha}_t)} \|x_t - \sqrt{\bar{\alpha}_t}x_0\|^2 + C$$

Taking the gradient with respect to x_t :

$$\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t}$$

Using the reparameterization trick $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$:

$$\nabla_{x_t} \log q(x_t|x_0) = -\frac{\sqrt{1 - \bar{\alpha}_t}\epsilon}{1 - \bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}}$$

Therefore, the score function $\mathbf{s}_{\theta}(x_t, t)$ approximates the gradient of the log-likelihood, which is directly proportional to the noise predicted by the DDPM model:

$$\mathbf{s}_{\theta}(x_t, t) \approx \nabla_{x_t} \log q(x_t) = \mathbb{E}_{q(x_0)}[\nabla_{x_t} \log q(x_t|x_0)] \approx -\frac{\epsilon_{\theta}(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

This reveals a fundamental connection: **predicting the noise (DDPM) is the same as estimating the score function (NCSN)**.

2.3 Denoising Diffusion Implicit Models

Despite the high generation quality of DDPM, they suffer from significant practical limitations that hindered their initial real-time deployment.

- **Slow Sampling Speed:** The fundamental assumption of DDPM is that the reverse process approximates a Gaussian distribution only when the step size is infinitesimal. Consequently, generation requires iterating through the full Markov chain (e.g., $T = 1000$ steps). As noted by Song et al. [12]: *For example, it takes around 20 hours to sample 50k images of size 32×32 from a DDPM, but less than a minute to do so from a GAN on an Nvidia 2080 Ti GPU.*
- **Stochastic Inconsistency:** In DDPM, the reverse process is stochastic ($x_{t-1} \sim \mathcal{N}(\mu_\theta, \sigma_\theta)$). This means that starting from the exact same initial noise x_T , running the sampling process twice will result in two different images. This lack of determinism makes it difficult to perform operations like interpolation or semantic manipulation in the latent space.

The connection between DDPM and NCSN provides a crucial insight. As established in the previous section, the noise prediction network $\epsilon_\theta(x_t, t)$ is mathematically equivalent to the score function (the gradient of the data density):

$$\epsilon_\theta(x_t, t) \propto -\nabla_{x_t} \log q(x_t)$$

This equivalence shifts our understanding of the generation process. Instead of viewing sampling solely as a “random walk” removing noise step-by-step, we can view it as moving along a vector field defined by the gradient.

- If we view the process as solving a **Stochastic Differential Equation (SDE)**, we are forced to take small, random steps (DDPM).
- However, if we view the score function as defining a deterministic flow, we can define an **Ordinary Differential Equation (ODE)** that shares the same marginal distributions.

This realization implies that the requirement to traverse T steps is an artifact of the specific Markovian formulation of DDPM, not a requirement of the diffusion objective itself. If we can define a deterministic trajectory (ODE) that connects noise to data, we can “skip steps” using numerical solvers, effectively jumping from $x_{1000} \rightarrow x_{950} \cdots \rightarrow x_0$.

Song et al. proposed **Denoising Diffusion Implicit Models (DDIM)** [12] by generalizing the forward process. They asked: *Can we define a non-Markovian forward process that leads to the same training objective as DDPM but allows for a deterministic reverse process?*

To understand how DDIM generalizes the sampling process, we must look at the construction of x_{t-1} given x_t and x_0 . In the standard DDPM framework, the forward process $q(x_t|x_0)$ is fixed. DDIM proposes a family of inference distributions $q_\sigma(x_{t-1}|x_t, x_0)$ that are non-Markovian but maintain the **Marginal Consistency Condition**:

$$q_\sigma(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t))$$

This condition is critical because it ensures that the training objective (MSE Loss on noise prediction) remains valid, allowing us to use pre-trained DDPM checkpoints.

We start with the fundamental definition of x_{t-1} conditioned on x_0 . By definition of the marginals, x_{t-1} can be written as:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_{t-1}$$

The innovation in DDIM is to split the noise term $\sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_{t-1}$ into two independent components:

- A component correlated with the current noise ϵ_t (deterministic direction).
- A new independent noise component ϵ (stochastic direction).

Let σ_t be the standard deviation of the new random noise. We can rewrite the equation by decomposing the variance $(1 - \bar{\alpha}_{t-1})$:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{(1 - \bar{\alpha}_{t-1}) - \sigma_t^2} \cdot \epsilon_t + \sigma_t \epsilon$$

In the reverse process, we do not know x_0 or the true noise ϵ_t . However, our neural network $\epsilon_\theta(x_t, t)$ provides an estimate of the noise. First, we express the estimated x_0 (denoised observation) from the current step x_t :

$$\text{predicted } x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t)}{\sqrt{\bar{\alpha}_t}}$$

Substituting this predicted x_0 and the predicted noise $\epsilon_\theta(x_t)$ back into the above equation, we obtain the **Generalized Sampling Formula**:

$$x_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{Denoised } x_0 \text{ term}} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \epsilon_\theta(x_t)}_{\text{Direction pointing to } x_t} + \underbrace{\sigma_t \epsilon}_{\text{Random Noise}}$$

This formulation implies that the reverse conditional distribution $q_\sigma(x_{t-1}|x_t, x_0)$ is a Gaussian distribution with the following mean and variance:

$$q_\sigma(x_{t-1}|x_t, x_0) = \mathcal{N} \left(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(x_t), \sigma_t^2 \right)$$

This equation breaks the sampling step into three components:

- **Predicted x_0 :** The model estimates the clean image from the current noisy input.
- **Direction to x_t :** This term accounts for the noise variance at step $t - 1$, ensuring the variance schedule is respected.
- **Random Noise:** Controlled by σ_t .

This derivation shows that by controlling the hyperparameter σ_t , we can interpolate between the stochastic DDPM process and the deterministic DDIM process.

- **DDPM Case:** If we set $\sigma_t = \tilde{\beta}_t = \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}}$, the equation collapses back to the original DDPM sampling.
- **DDIM Case:** If we set $\sigma_t = 0$, the random noise term vanishes completely. The sampling process becomes deterministic:

$$x_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t)$$

This deterministic mapping implies that for every initial noise x_T , there is a unique corresponding output x_0 . Furthermore, because this process approximates an ODE, the error from discretization is smaller, allowing us to skip steps (e.g., update every 20 or 50 steps) without significantly degrading image quality. This reduces inference time from thousands of steps to just 20-50 steps.

2.4 Rectified Flow and Flow Matching

2.4.1 Velocity Prediction

In the original DDPM formulation, the model is trained to predict the noise ϵ added to the image. However, as diffusion models evolved, researchers identified significant numerical instabilities with this parameterization, particularly when the noise level is high. To address this, Salimans & Ho introduced **Velocity Prediction** [13], which has since become the standard for advanced models.

Recall the forward diffusion process equation:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

To recover the original image x_0 from the prediction, we must invert this equation:

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

A critical issue arises at the end of the forward process ($t \rightarrow T$), where the signal is entirely destroyed. At this stage, $\bar{\alpha}_t \rightarrow 0$ and the image is pure noise.

- **ϵ -prediction:** The model tries to predict ϵ from x_t (which is effectively ϵ). The reconstruction of x_0 involves dividing by $\sqrt{\bar{\alpha}_t} \approx 0$. As shown in the provided materials, this leads to a division by zero singularity:

$$x_0 = \frac{x_t - \epsilon}{0} \rightarrow \infty \quad (\text{Unstable})$$

- **v -prediction:** The model predicts a velocity vector v_t . In the limit where $\alpha_t \rightarrow 0$, the relationship simplifies to $x_0 = -v_t$, which remains numerically stable and bounded.

The concept of “velocity” comes from interpreting the diffusion process as a rotation in a high-dimensional space between the data x_0 and the noise ϵ . If we parameterize the mixing coefficients trigonometrically such that $\sqrt{\bar{\alpha}_t} = \cos(\phi_t)$ and $\sqrt{1 - \bar{\alpha}_t} = \sin(\phi_t)$, the process describes a circular trajectory:

$$x_t = \cos(\phi_t)x_0 + \sin(\phi_t)\epsilon$$

The velocity v_t is defined as the rate of change of x_t with respect to the angular time ϕ_t :

$$v_t = \frac{dx_t}{d\phi_t} = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1 - \bar{\alpha}_t}x_0$$

This definition creates a target that combines both the noise and the clean image. This implies that the model learns to predict a weighted mixture of the signal and the noise, ensuring that the target is meaningful at all noise levels.

During inference, if the model predicts v_t , we can analytically recover both the denoised image x_0 and the noise ϵ by solving the linear system formed by the definitions of x_t and v_t :

$$\begin{cases} x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \\ v_t = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1 - \bar{\alpha}_t}x_0 \end{cases}$$

Solving this system yields the reconstruction formulas:

$$\begin{cases} x_0 = \sqrt{\bar{\alpha}_t}x_t - \sqrt{1 - \bar{\alpha}_t}v_t \\ \epsilon = \sqrt{1 - \bar{\alpha}_t}x_t + \sqrt{\bar{\alpha}_t}v_t \end{cases}$$

2.4.2 Rectified Flow

While Diffusion Models established a powerful generative baseline, their sampling efficiency was historically bottlenecked by the curvature of their generative trajectories. **Rectified Flow** [7] addresses this by fundamentally reimagining the transport map between the noise distribution and the data distribution.

In standard Variance Preserving Diffusion Models (like DDPM), the forward process is defined as:

$$x_t = \cos(\phi_t)x_0 + \sin(\phi_t)\epsilon$$

Geometrically, because $\cos^2(\phi) + \sin^2(\phi) = 1$, the interpolation coefficients ensure that the norm of the data remains constant (assuming normalized inputs). Consequently, the transport trajectory from data to noise lies on a high-dimensional **hypersphere**. Generating an image requires the ODE solver to traverse this curved spherical path. A curved path is inherently longer and more complex to approximate numerically than a straight line, necessitating many discretization steps (e.g., 50-100 steps) to avoid truncation errors.

Rectified Flow simplifies this by forcing the transport trajectory to lie on a flat **hyperplane**. It defines the forward process as a simple linear interpolation:

$$x_t = (1 - t)x_0 + t\epsilon, \quad t \in [0, 1]$$

Here, $t = 0$ corresponds to the clean data x_0 , and $t = 1$ corresponds to pure noise ϵ . This defines the shortest possible path (a straight line) between the two distributions.

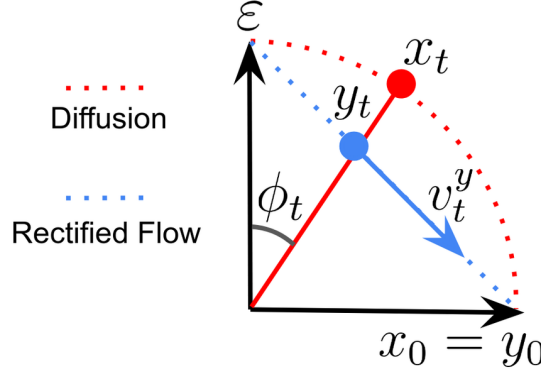


Figure 4: Relationship of Diffusion and Rectified Flow

To implement this linear trajectory, the model learns a velocity field v_t that predicts the time derivative of the state x_t . Differentiating the linear equation with respect to t yields the target velocity:

$$v_t = \frac{dx_t}{dt} = \frac{d}{dt}((1-t)x_0 + t\epsilon) = \epsilon - x_0$$

Thus, the neural network $v_\theta(x_t, t)$ is trained to predict the difference vector $(\epsilon - x_0)$ given the interpolated state x_t . During sampling, we solve the ODE: $dx = v_\theta(x_t, t)dt$ starting from noise. Using a simple Euler solver, the update rule becomes:

$$x_{s_1} = x_{s_2} + (s_2 - s_1)v_\theta(x_{s_2}, s_2)$$

Theoretically, if the velocity field is perfectly learned, one could generate an image in a single Euler step ($T \rightarrow 1$). However, due to prediction errors, practical implementations often use a few steps (e.g., 4 steps) to achieve high fidelity.

A critical challenge in training Rectified Flow is the **Crossing Paths** problem:

- **Cause:** In the initial training phase (1-Rectified Flow), we randomly pair a data point x_0 (e.g., an image of a cat) with an independent noise sample ϵ . We then ask the model to learn a straight line connecting them.
- **Conflict:** Because the pairing is random, the trajectory connecting (x_0^A, ϵ^A) might intersect with the trajectory of (x_0^B, ϵ^B) in the high-dimensional space.
- **Consequence:** ODE trajectories must be unique and cannot intersect. When the model is forced to learn intersecting paths, it learns the average of the conflicting velocity fields. This results in a learned vector field that is not straight but curved and chaotic, undermining the “straight line” objective and degrading generation quality.

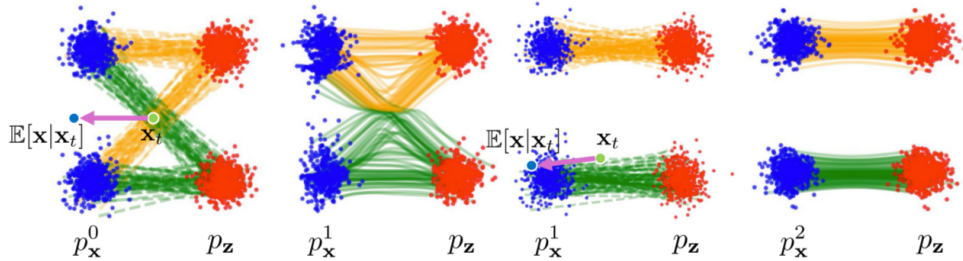


Figure 5: Straightening Paths using Reflow

To resolve this, Liu et al. (2022) introduced a recursive training procedure called **Reflow**. The Reflow algorithm works by straightening the trajectories iteratively:

- **Stage 1 (1-Rectified Flow):** Train the model on random pairs (x_0, ϵ) . This yields a crude model v_θ^1 .

- **Synthetic Data Generation:** Use the trained ODE v_θ^1 to generate pairs. Specifically, start from noise ϵ and simulate the flow to reach a generated image $\hat{x}_0$. Importantly, this ODE trajectory is deterministic and non-intersecting by definition.
- **Stage 2 (Reflow):** Train a new model v_θ^2 on the pairs $(\hat{x}_0, \epsilon)$ generated in the previous step. Since these pairs are connected by the ODE paths of the first model, the new paths are naturally “straightened” and uncrossed.

Empirically, this “rewiring” of the noise-to-data couplings significantly reduces the transport cost and allows for high-quality few-step generation.

2.4.3 Flow Matching

Flow Matching [8] serves as a generalized theoretical framework that encompasses various generative approaches, including Diffusion Models and Rectified Flow.

Instead of being tied to a specific noise schedule or probabilistic formulation, Flow Matching defines the generative process simply as learning a **Vector Field** (or velocity field) that transports a source probability distribution p_0 (noise) to a target distribution p_1 (data). This framework allows us to view different models through a geometric lens:

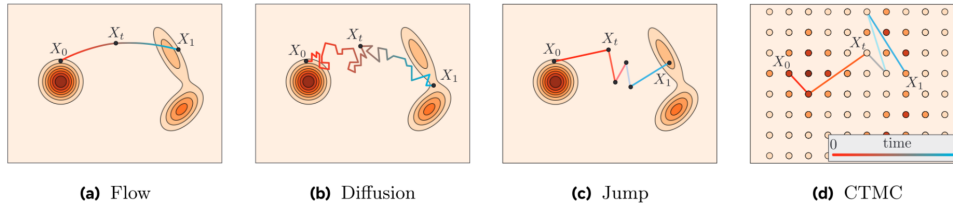


Figure 6: Flow Matching Guide

- **Diffusion Models (Curved Path):** Standard diffusion models (like DDPM) can be interpreted as a specific instance of Flow Matching where the probability path follows a curved trajectory on a high-dimensional **hypersphere**. This curvature arises from the constraint of preserving the variance during the forward process.
- **Rectified Flow (Straight Path):** As discussed in the previous section, Rectified Flow defines the transport path as a straight line on a flat **hyperplane**, representing the optimal transport coupling between noise and data.

The core contribution of Flow Matching is the **Conditional Flow Matching** (CFM) objective, which provides a tractable regression loss to learn the vector field for **any** desired probability path. This unifies the field, demonstrating that “predicting noise” (in Diffusion) and “predicting velocity” (in Rectified Flow) are essentially different coordinate systems for the same fundamental task of learning a flow from chaos to structure.

2.5 Rectified Diffusion

By 2020, DDPM had established themselves as the dominant paradigm in generative AI. Massive resources were poured into training Diffusion Models, which required months of GPU compute and vast datasets. However, the emergence of Flow Models (Rectified Flow and Flow Matching) in late 2022 and 2023 introduced a superior theoretical framework with straight-line trajectories, offering significantly faster and more deterministic sampling.

This created a significant challenge: *Should the community discard the powerful, well-trained diffusion models weights to train flow models from scratch?* Doing so would render the extensive “knowledge” embedded in models, wasting millions of dollars in computational costs.

To solve this, researchers proposed **Rectified Diffusion**, a strategy to upgrade existing diffusion models into flow models rather than training new ones from scratch. The core idea is to apply the Reflow procedure directly to a pre-trained diffusion checkpoint.

The transformation typically involves two key conceptual shifts:

- **Objective Alignment** ($\epsilon \rightarrow v$): Since standard diffusion models often predict noise (ϵ), while flow models predict velocity (v), the model is first fine-tuned or adapted to predict velocity ($v_t = \epsilon - x_0$). This re-parameterization allows the diffusion model to be interpreted as an Ordinary Differential Equation (ODE) generator.
- **Reflow Distillation**: Once interpreted as a flow, the model undergoes the Reflow process. The model generates synthetic data pairs (z, x) using its original curved ODE trajectory. It is then retrained on these pairs to “straighten” the path connecting the noise z to the image x .

2.5.1 InstaFlow

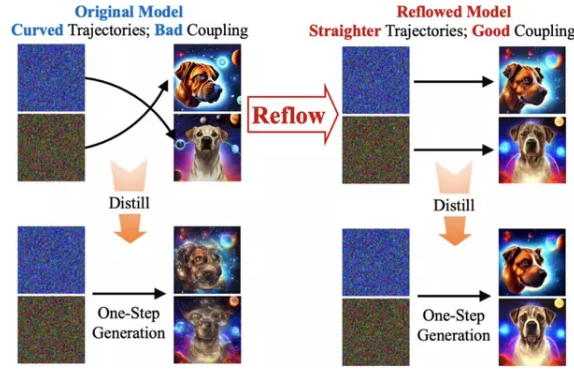


Figure 7: InstaFlow - One Step is Enough

A prominent implementation of this approach is **InstaFlow** [14]. Instead of training a new architecture, the authors took the pre-trained standard diffusion model with curved stochastic paths and applied text-conditional Reflow. By forcing the model to learn a straight flow between the noise distribution and the image distribution, InstaFlow successfully compressed the inference process from 25-50 steps down to just **1 step** while maintaining high image quality. This demonstrated that the benefits of Flow Matching could be retroactively applied to the previous generation of diffusion models, bridging the gap between the two eras.

2.5.2 PeRFlow

While InstaFlow demonstrated that diffusion models could be converted into rapid one-step generators, it faced a fundamental limitation: **Global Truncation Error**. Attempting to straighten the entire curved trajectory of a diffusion ODE into a single straight line often results in significant approximation errors, leading to blurred images or loss of detail unless extensive synthetic datasets are used.

To address this, **PeRFlow** (Piecewise Rectified Flow) [15] proposes a “Divide-and-Conquer” strategy. Instead of forcing the entire path from noise z_1 to data z_0 to be a single straight line, PeRFlow divides the time horizon into multiple windows and straightens the trajectory within each window independently. This results in a **piecewise linear** trajectory (a “polyline”) rather than a single straight line, significantly reducing the truncation error while retaining the speed benefits of flow models (typically requiring only 4 steps).

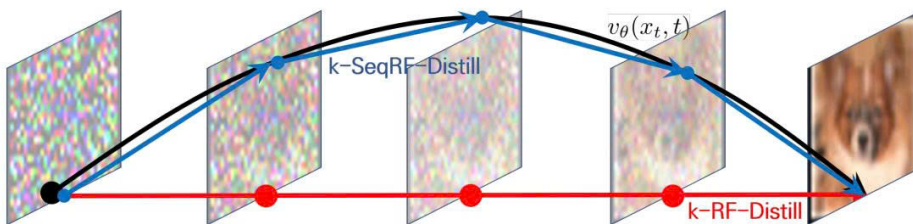


Figure 8: Sequential Flow Straightening

It is also important to note that this conceptual breakthrough was reached simultaneously by two independent research groups. **SeqRF** (Sequential Rectified Flow) [16] and **PeRFlow** were published around the same time, sharing the core mathematical intuition of segmented flow straightening. While SeqRF focuses heavily on the theoretical bounds of the truncation error, PeRFlow emphasizes the “Plug-and-Play” capability compatible with Stable Diffusion workflows.

3 Model Architectures

Having established the theoretical foundations of Diffusion Models and Flow Matching in the previous chapter, we now focus on the practical realization of these concepts: the **Neural Network Architectures**.

While the mathematical frameworks define the training objective (e.g., minimizing the error in noise or velocity prediction), the architecture determines the capacity, expressivity, and efficiency of the generative process. Fundamentally, the role of the model is to function as a universal approximator $f_\theta(x_t, t, c)$, which takes a noisy input x_t , a timestep t , and a conditioning signal c (such as a text prompt) to predict the denoising direction.

Modern SOTA T2I systems are rarely monolithic. Instead, they are composite systems designed to handle the high dimensionality of image data and the complexity of semantic understanding. This chapter dissects the anatomy of these systems into three critical components:

- **Denoising Backbone:** The core neural network responsible for the iterative generation process.
- **Conditioning Mechanisms:** The modules that project human intent (text, class labels) and temporal information into the visual generation process.
- **Perceptual Compression:** The use of Variational Autoencoders (VAEs) to translate high-dimensional pixel space into a computationally manageable latent space.

3.1 Denoising Backbone

3.1.1 U-Net

The **U-Net architecture** [17], originally introduced by Ronneberger et al. for biomedical image segmentation, serves as the dominant backbone for the majority of first-generation diffusion models.

Fundamentally, the U-Net is a specialized Convolutional Neural Network (CNN) designed for image-to-image (I2I) translation tasks where the output spatial resolution matches the input. Its name derives from its symmetric “U-shaped” structure, consisting of a contracting path (Encoder) and an expansive path (Decoder). The classic U-Net architecture is composed of three distinct components:

- **Downsampling Stack (Encoder):** This path captures the context and extracts high-level semantic features. It consists of repeated applications of two 3×3 unpadded convolutions, each followed by a Rectified Linear Unit (ReLU) and a 2×2 max pooling operation with stride 2 for downsampling. At each downsampling step, the spatial resolution is halved, while the number of feature channels is typically doubled to preserve information capacity.
- **Upsampling Stack (Decoder):** This path enables precise localization. Each step involves upsampling the feature map (often via transposed convolution or nearest-neighbor interpolation) followed by a 2×2 convolution that halves the number of feature channels.
- **Skip Connections (Shortcuts):** A defining innovation of the U-Net is the use of long-range skip connections. These connections concatenate the high-resolution feature maps from the Encoder directly to the corresponding layers in the Decoder. This is critical for diffusion models because the noise prediction task requires identifying fine-grained, high-frequency details that are often lost during aggressive downsampling.

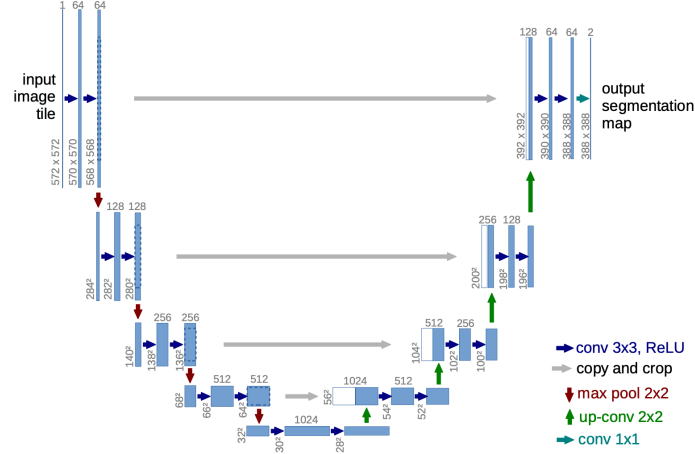


Figure 9: U-net architecture

The selection of U-Net as the backbone for DDPM was not arbitrary but driven by several historical successes in generative deep learning:

- **Pix2Pix Success:** The Pix2Pix GAN [18] framework demonstrated that U-Net-based generators were vastly superior to standard encoder-decoder architectures for I2I translation tasks, effectively handling the preservation of structural integrity.
- **Medical Imaging Dominance:** The U-Net’s ability to learn from limited data and produce highly accurate segmentation maps has made it the de facto standard in medical imaging competitions, proving its robustness and efficiency.
- **PixelCNN++ Influence:** The specific variant of U-Net used in diffusion models is heavily influenced by PixelCNN++ [19]. It integrates **ResNet blocks** [20] into the U-Net depth to improve gradient flow and training stability.

While the macro-structure remains U-shaped, the U-Net used in modern diffusion models differs significantly from the original to accommodate the specific requirements of noise prediction:

- **ResNet Blocks:** Instead of simple stacked convolutions, each level of the U-Net employs Residual Blocks composed of Group Normalization [21], SiLU activation [22], and Convolutional layers. This allows for deeper networks without the vanishing gradient problem.
- **Time Embedding Injection:** Since the noise prediction $\epsilon_\theta(x_t, t)$ depends heavily on the noise level t , the timestep is encoded and projected into a vector. This time vector is then injected into every ResNet block in the U-Net (via addition or scale-and-shift mechanisms), ensuring the model knows “how noisy” the image currently is.
- **Self-Attention Layers:** To capture long-range dependencies that convolutions (with their local receptive fields) might miss, modern Diffusion U-Nets insert Self-Attention mechanisms at lower spatial resolutions (e.g., 16×16 or 8×8).

3.1.2 Diffusion Transformer

While the U-Net architecture dominated the early landscape of diffusion models, the **Diffusion Transformer** (DiT) [23], introduced by Peebles and Xie, represents a fundamental paradigm shift. DiT replaces the traditional convolutional U-Net backbone with a standard Transformer architecture.

The transition from U-Net to DiT was motivated by several key observations in the broader deep learning community:

- **Vision Transformers (ViT) Success:** Dosovitskiy et al. [24] demonstrated that the inductive biases of convolutions (local receptive fields, translation invariance) are not strictly necessary for vision tasks if sufficient data and compute are available. ViTs successfully applied the standard NLP Transformer to images.

- **Predictable Scaling Laws:** Transformer architectures are uniquely renowned for their scalability. Unlike CNNs, which can plateau or become unstable at extreme depths, Transformers reliably exhibit improved performance as model size and compute (FLOPs) increase.
- **Unified Architecture:** By abandoning the complex downsampling/upsampling cascades and asymmetric skip connections of the U-Net, DiT offers a conceptually simpler, homogeneous architecture that is easier to optimize and parallelize on modern hardware.

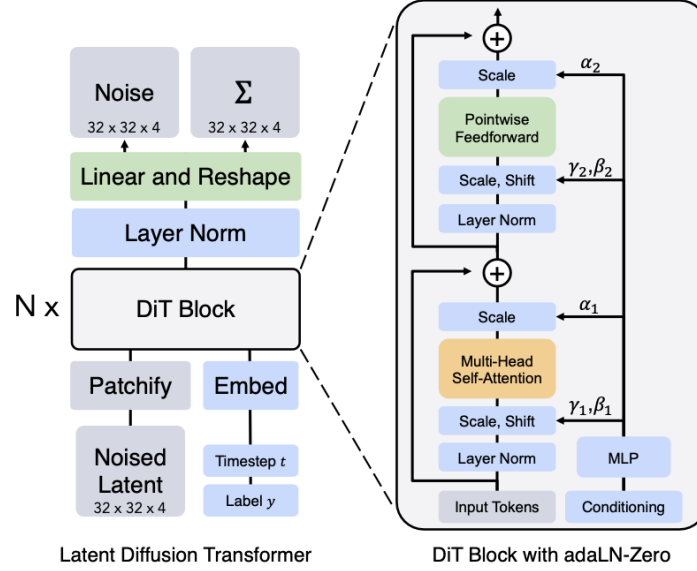


Figure 10: Diffusion Transformer (DiT) architecture

The DiT architecture closely follows the original ViT formulation but is adapted for the generative denoising task. The forward pass consists of the following steps:

- **Patchification:** Instead of processing the input as a full spatial grid, DiT takes the spatial latent representation of an image (e.g., a $32 \times 32 \times 4$ tensor from a VAE) and “patchifies” it. The spatial latent is divided into non-overlapping patches of size $p \times p$.
- **Sequence Conversion:** These patches are linearly projected and flattened into a 1D sequence of tokens of dimension d . Standard frequency-based positional embeddings (like sine-cosine embeddings) are added to retain spatial awareness before the sequence is passed to the Transformer blocks.
- **Transformer Blocks:** The sequence of tokens is processed through a series of identical multi-head self-attention and feed-forward layers. Because the spatial resolution is not downsampled, the sequence length remains constant throughout the network.
- **Output Decoding:** After the final Transformer block, a linear decoder projects the token sequence back into the original spatial latent shape. The network is configured to output both the noise prediction (ϵ_θ) and a diagonal covariance prediction (σ_θ) for the reverse diffusion step.

3.2 Conditioning Mechanisms

The fundamental premise of the standard diffusion process, as formulated in early works, is the **Unconditional Diffusion Model**. In this paradigm, the network learns the marginal probability distribution $P(x)$ of the training data. The objective is to generate an arbitrary sample x belonging to the dataset by predicting the noise $\epsilon(x_t, t)$. While mathematically elegant, unconditional generation lacks user control.

To make generative AI practically useful, the field shifted towards **Conditional Diffusion Models**. These models learn the conditional probability distribution $P(x|y)$, where y represents an external guiding signal. The network’s objective is modified to generate a specific sample x that accurately matches

the provided condition y , altering the prediction function to $\epsilon(x_t, t, y)$. Depending on the nature of the condition y , several prominent paradigms have emerged:

- **Class-Conditional Generation:** The model synthesizes an image based on a discrete categorical label (e.g., generating a “dog” vs. a “cat” from the ImageNet dataset).
- **Text-to-Image (T2I):** The model receives a natural language prompt and synthesizes a corresponding image from pure noise. This requires profound semantic alignment between language and vision.
- **Image-to-Image (I2I):** The model takes an existing image and a text prompt as inputs, utilizing the network to modify or translate the visual content based on the text. While it may superficially appear as a mere sub-branch of T2I since both rely on textual guidance, the fundamental difference lies in the initial state of the generative trajectory. Whereas T2I initiates the denoising process from pure, unstructured Gaussian noise, I2I typically begins with a partially noised version of a reference image (acting as a base). This provides the network with a strong structural and spatial prior (such as composition, geometry, and color distribution) that a text prompt alone cannot enforce, making the conditioning multi-modal.

While this report heavily emphasizes the T2I domain, all conditional generation relies on specific architectural interventions. **Conditioning Mechanisms** is the technical term for the precise methods, modules, and mathematical operations used to inject this external guiding information (such as text, labels, or even the mandatory temporal step t) into the layers of the denoising backbone. Without these mechanisms, the U-Net or DiT would process the noisy image blindly.

The following sections detail the crucial components required to encode human intent and temporal states, and how they are physically wired into the generative backbone.

3.2.1 Text Encoders

Natural language serves as the primary and most intuitive interface for modern generative AI. However, the denoising backbone (whether U-Net or DiT) operates on continuous numerical tensors and cannot process raw text directly. The role of a **Text Encoder** is to bridge this gap by translating discrete human language into dense, high-dimensional semantic representations (embeddings) that guide the generation process. In the context of T2I diffusion models, two predominant families of text encoders have shaped the landscape:

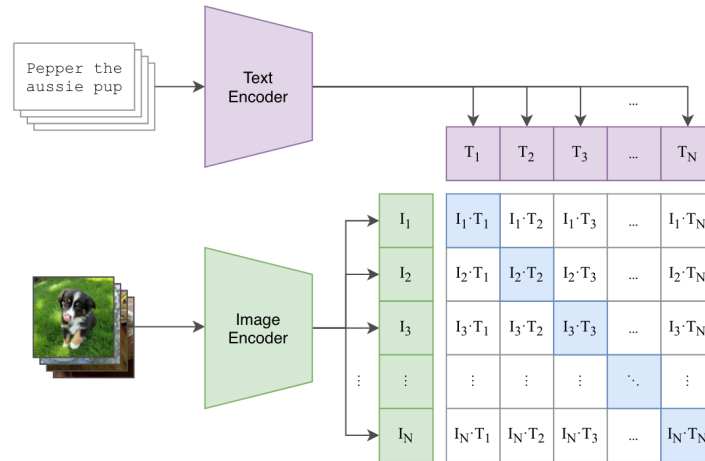


Figure 11: CLIP approach

- **CLIP (Contrastive Language-Image Pre-training):** Developed by OpenAI, CLIP [25] serves as the foundational text encoder for the majority of first-generation diffusion models. The power of CLIP lies in its multimodal training objective. Instead of being trained solely to predict the next word (like standard language models), CLIP is trained on hundreds of millions of image-text pairs to understand the relationship between visual and textual concepts. The training process involves

two separate networks: an Image Encoder (usually a Vision Transformer) and a Text Encoder (a standard Transformer). During pre-training, a batch of matched image-text pairs is processed simultaneously. The model employs a **contrastive loss function**, which mathematically pushes the embeddings of matching image-text pairs closer together in a shared latent space (maximizing cosine similarity) while pushing apart the embeddings of mismatched pairs. This results in a text encoder that possesses a profound structural understanding of how words map to visual features.

- **T5 (Text-to-Text Transfer Transformer):** While CLIP excels at capturing general semantic concepts (e.g., an “apple” or a “car”), its relatively small size and contrastive objective limit its ability to comprehend complex syntax, precise spatial relationships (e.g., “a red cube behind a blue sphere”), or exact text rendering. To address these limitations, next-generation diffusion models integrate massive, pure-language models like **T5-XXL** [26]. Because T5 is trained on vast corpora of text to understand deep linguistic structures, it provides much richer and highly detailed embeddings, dramatically improving the model’s prompt adherence and ability to follow complex compositional instructions.

Regardless of the specific architecture (CLIP or T5), the pipeline for converting a user’s prompt into a mathematical condition involves several standardized steps:

- **Tokenization:** The raw text string is broken down into discrete sub-word units called tokens using algorithms like Byte-Pair Encoding (BPE) [27]. These tokens are mapped to numeric IDs based on a fixed vocabulary.
- **Sequence Embedding:** The token IDs are passed through the text encoder’s embedding layer and subsequent transformer blocks. The sequence is typically padded or truncated to a fixed maximum length, L (e.g., 77 tokens in standard CLIP).
- **Hidden State Extraction:** Unlike classification tasks that use a single “pooled” vector to represent the entire sentence, diffusion models typically extract the *unpooled hidden states* from the final or penultimate layer of the text encoder.

The output is a sequence of feature vectors mathematically represented as a tensor of shape $\mathbb{R}^{L \times D}$, where L is the sequence length and D is the embedding dimension (e.g., 77×768 , or much larger for T5). This matrix of token-level embeddings provides fine-grained, word-by-word semantic context, which is then fed directly into the denoising backbone to steer the spatial generation of the image.

3.2.2 Timestep Embeddings

A fundamental characteristic of diffusion models is that a single neural network is utilized to perform the denoising steps across the entire reverse trajectory. However, the nature of the denoising task varies drastically depending on the current phase of the process. For instance, predicting the noise at step $t = 1000$ (where the image is practically indistinguishable from pure Gaussian noise) requires entirely different feature extraction and scaling compared to step $t = 1$ (where the image is nearly clean and only requires minor high-frequency refinement). Because the network weights are shared across all timesteps, the model must be explicitly informed of the current timestep t to adapt its internal behavior and feature processing accordingly.

To solve this temporal awareness problem, Ho et al. borrowed the concept of **Positional Encoding** [28] from the NLP Transformer architecture. Instead of feeding the scalar integer t directly into the network—which scales poorly and fails to capture relational distances smoothly—the timestep is first projected into a high-dimensional continuous vector space using sinusoidal functions of varying frequencies.

For a given timestep t and a desired embedding dimension d , the encoding is defined mathematically as:

$$PE_{(t,2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right)$$

$$PE_{(t,2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

where i represents the dimension index. This fixed trigonometric projection allows the model to easily learn and extrapolate relative temporal positions, ensuring that adjacent diffusion steps (e.g., $t = 500$ and $t = 499$) have highly similar and continuous embedding representations.

The raw sinusoidal embedding is structurally useful but is not injected directly into the spatial layers of the network. Instead, it serves as the foundational input to a dedicated Multi-Layer Perceptron (MLP) module. This temporal MLP typically consists of two linear (dense) layers separated by a non-linear activation function, such as SiLU (Swish) or Mish [29].

While the initial sinusoidal projection is fixed and mathematically deterministic, the weights of this subsequent MLP are **learned** during the training phase. This hybrid approach enables the network to map the rigid mathematical frequencies into a flexible, domain-specific semantic representation of time that is optimally tuned for the dataset’s specific denoising trajectory.

Once the timestep t is transformed into a dense, learned vector embedding, it must be broadcasted throughout the entire denoising backbone so that every layer is temporally aware.

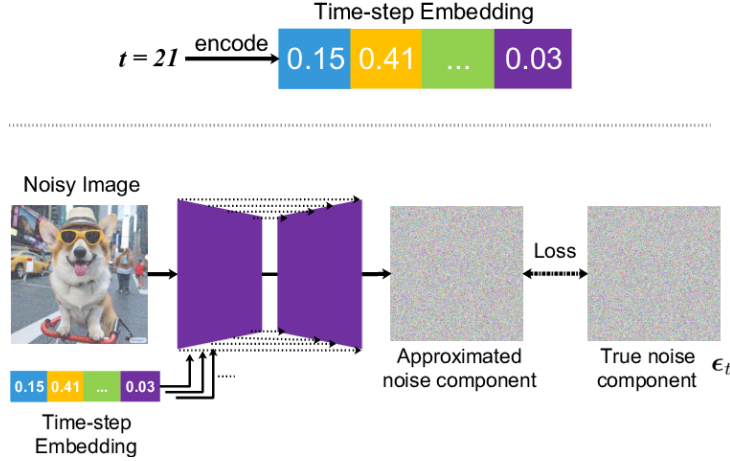


Figure 12: Timestep Embedding

3.2.3 Adaptive Normalization

Once the conditioning signals (such as text prompts and timesteps) are converted into dense vector embeddings, the next architectural challenge is integrating them into the spatial feature maps of the denoising backbone. While simple element-wise addition or channel concatenation are viable methods, they often lack the expressivity needed for complex semantic control. To address this, modern diffusion models rely heavily on **Adaptive Normalization**.

Adaptive Normalization (AdaNorm) variations were first introduced into Diffusion architectures around May 2021 specifically to handle Class Label and Time step information. The core motivation is that external conditions should not merely be appended to the features, they should fundamentally modulate the statistical distribution of the feature maps. By shifting and scaling normalized activations, the network acts like a set of dynamic filters, amplifying or suppressing certain visual features based on the temporal state or the semantic prompt.

The mechanism operates in two distinct phases. First, the input spatial feature map x undergoes a standard normalization process to achieve a zero mean and unit variance:

$$x_{norm} = \frac{x - \mu}{\sigma}$$

Here, μ and σ are the mean and standard deviation computed over specific dimensions depending on the base normalization technique (e.g., across a group of channels in Group Normalization or across the embedding dimension in Layer Normalization [30]).

Second, this normalized feature map is subjected to an affine transformation (scaling and shifting) that is directly controlled by the conditioning signals:

$$output = \gamma(y, t)x_{norm} + \beta(y, t)$$

Unlike standard normalization layers where the scale (γ) and shift (β) are learned as static global weights, Adaptive Normalization generates these parameters dynamically on the fly.

To achieve this, the conditioning inputs are first vectorized. The condition vector (such as a class label or text embedding) can be added or concatenated with the embedding vector of step t . This combined

conditional representation is then passed through one or more Linear (dense) layers. The outputs of these Linear layers are the exact values used for the scale (γ) and shift (β) of the Normalization layer.

In practice, within a U-Net’s ResNet block, the features are normalized via GroupNorm, and then the predicted γ and β from the timestep/text MLP are applied before the subsequent convolution. In Diffusion Transformers (DiT), this exact mathematical principle is implemented as Adaptive Layer Normalization (adaLN), where the γ and β parameters modulate the token sequences immediately prior to the Self-Attention and Feed-Forward sub-blocks, ensuring that the contextual guidance deeply penetrates every computational step of the generation process.

3.2.4 Cross Attention

While Adaptive Normalization is highly effective for injecting global scalars or singular vectors (such as the diffusion timestep or a discrete class label), it lacks the spatial granularity required to map complex, sequential data-like a full sentence of text—to specific regions of an image. To achieve fine-grained, localized semantic control, modern T2I architectures rely heavily on the **Cross-Attention** mechanism. Within the transformer blocks embedded in modern diffusion backbones (both U-Net and DiT), attention mechanisms serve two distinct but complementary purposes:

- **Self-Attention:** The spatial feature map attends to itself. Every pixel (or latent patch) calculates its relationship with every other pixel to understand the internal spatial context and enforce global structural coherence.
- **Cross-Attention:** The spatial feature map attends to an external data modality (typically text). In this mechanism, each pixel “looks” at the sequence of word tokens to determine what specific visual content it needs to render at its spatial location based on the prompt.

The Cross-Attention operation is built upon the standard Scaled Dot-Product Attention [28]. It operates on three projection matrices: Queries (**Q**), Keys (**K**), and Values (**V**). The critical distinction from Self-Attention lies in the diverse origins of these matrices:

- **Queries (Q):** Derived from the current spatial representation of the image. The intermediate feature map of the U-Net (or DiT) is flattened into a sequence of spatial tokens, $\mathbf{Z} \in \mathbb{R}^{N \times d_z}$, and linearly projected to form $\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q$.
- **Keys (K) and Values (V):** Derived from the external conditioning signal. The output sequence from the Text Encoder (e.g., CLIP or T5), denoted as $\mathbf{C} \in \mathbb{R}^{L \times d_c}$ (where L is the number of text tokens), is linearly projected to form $\mathbf{K} = \mathbf{C}\mathbf{W}_K$ and $\mathbf{V} = \mathbf{C}\mathbf{W}_V$.

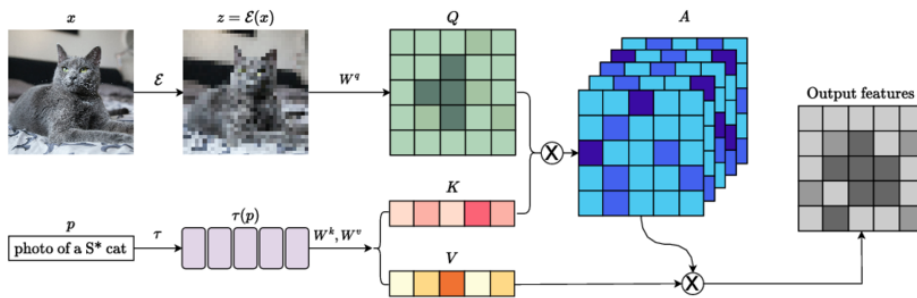


Figure 13: Single-head Cross-Attention

The cross-attention output is computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

The matrix multiplication $\mathbf{Q}\mathbf{K}^T$ calculates the dot product between every spatial patch and every text token. This results in an $N \times L$ attention map, which effectively represents a semantic alignment score: it measures how much each specific region of the image “cares” about each specific word in the prompt. The softmax function normalizes these scores into a probability distribution. Finally, multiplying by $\mathbf{V}$

computes a weighted sum of the text features based on this attention map. The output is a matrix of text-conditioned visual features, which is then projected and added back to the original image features $\mathbf{Z}$ via a residual connection.

Because computing attention is a highly memory-intensive operation ($O(N \cdot L)$ in the case of cross-attention and $O(N^2)$ for self-attention), it is often bottlenecked by high resolutions. In standard U-Net architectures, Cross-Attention (and Self-Attention) blocks are exclusively injected into the deepest layers of the network (e.g., 16×16 and 8×8 spatial resolutions), where the feature maps have been significantly downsampled. At these low-resolution bottlenecks, the model integrates the high-level semantic meaning of the text prompt, which subsequently guides the structural synthesis during the upsampling decoder path.

3.3 Perceptual Compression

While the denoising backbone and conditioning mechanisms define **how** a model learns to generate, the domain in which they operate dictates the **efficiency** of the process. Early diffusion models (such as DDPM and NCSN) operated directly in the **Pixel Space**. For an image of resolution 512×512 with 3 color channels, the network must predict noise for a tensor of 786,432 dimensions at every single timestep. This high-dimensional optimization is computationally exorbitant, extremely slow, and severely limits the generation of high-resolution images.

To circumvent this computational bottleneck, Rombach et al. [3] introduced the concept of **Perceptual Compression** through the **Latent Diffusion Model** (LDM) framework. The core insight of LDMs is that natural images contain a vast amount of high-frequency, imperceptible details. Evaluating the likelihood of these microscopic pixel variations demands immense computational resources but contributes negligibly to the overall semantic quality and human perception of the image. Therefore, the generative modeling process can be logically split into two distinct stages:

- **Perceptual Compression:** A specialized model removes high-frequency, visually redundant information, mapping the image into a mathematically dense, lower-dimensional space.
- **Semantic Generation:** The core generative model (U-Net or DiT) operates exclusively within this compressed space to learn the actual data distribution, composition, and semantic conditioning.

The workhorse of perceptual compression in LDMs is the **Variational Autoencoder** (VAE). Before the diffusion model is even trained, the VAE is trained independently on a massive dataset of images to reconstruct its inputs as accurately as possible.

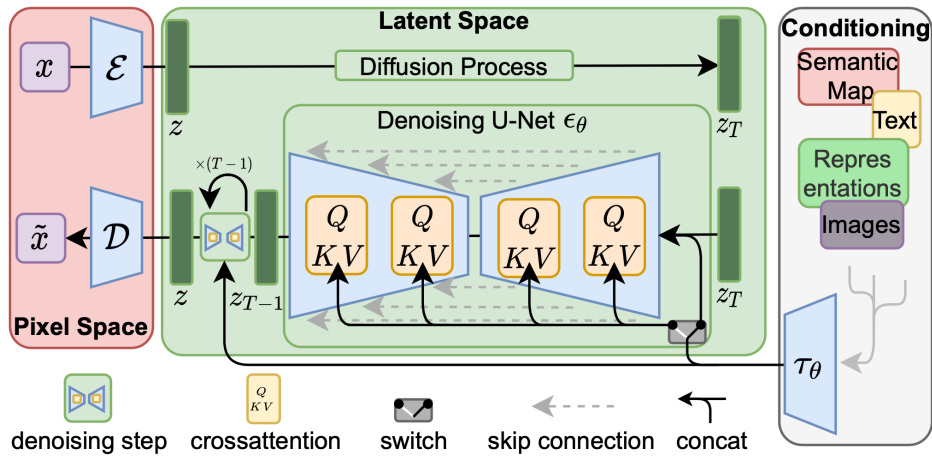


Figure 14: Latent Diffusion Model architecture

It is worth noting that not all SOTA models utilize perceptual compression. Pixel-space models, such as Google’s Imagen [31] or DeepFloyd IF [32], avoid latent spaces by employing a **Cascaded Architecture**. They generate a low-resolution image (e.g., 64×64) directly in pixel space and sequentially pass it through multiple independent, heavily parameterized super-resolution diffusion models to reach 1024×1024 . However, LDMs remain the dominant paradigm due to their superior balance of synthesis quality and consumer-grade hardware accessibility.

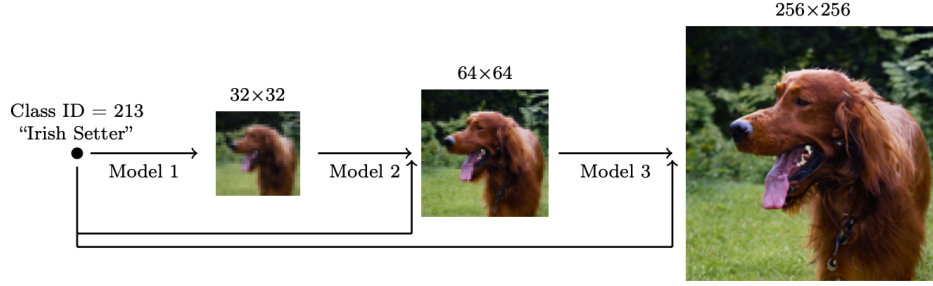


Figure 15: A cascaded pipeline of multiple diffusion models

3.4 Guidance Methods

While **Conditioning Mechanisms** are architectural modifications designed to inject conditional information during the training phase to help the network predict noise, they are sometimes insufficient on their own during inference.

During the sampling process, a standard conditional diffusion model often exhibits a tendency to “average out” its predictions. Because the model is trained to minimize the overall data likelihood, it leans heavily on its prior baseline knowledge of the dataset. Consequently, it may produce safe, generic images while ignoring the fine-grained, specific, or highly complex details present in a user’s prompt.

To counteract this, researchers developed **Guidance Methods**. These are primarily sampling-time interventions mathematically designed to manipulate the vector field or noise prediction, actively forcing the generative trajectory toward regions of the latent space that most strictly align with the provided condition.

3.4.1 Classifier Guidance

Classifier Guidance [33] was the first major breakthrough in controlling diffusion models, allowing them to beat GANs in image synthesis quality. This method relies on combining two distinct models operating simultaneously during the generation process: the core Diffusion Model and a separate, pre-trained Classifier.

At each denoising step t , the noisy image x_t is passed through an external classifier (e.g., an ImageNet classifier) that has been specifically trained to recognize noisy images. The classifier evaluates x_t and computes the gradient of the log probability of the target class y with respect to the image: $\nabla_{x_t} \log p_\phi(y|x_t)$.

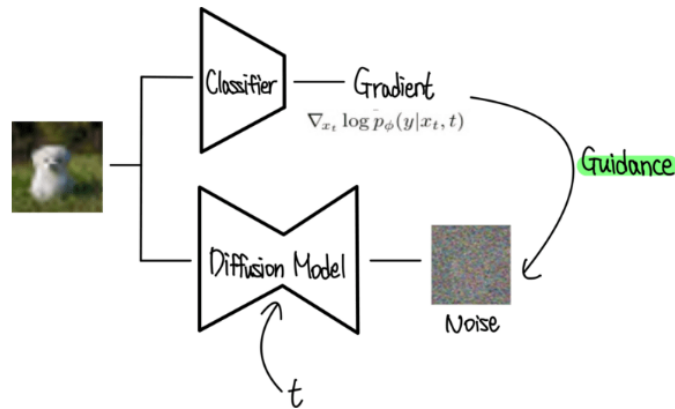


Figure 16: Classifier Guidance

This gradient acts as a directional vector pointing toward the exact features of class y . It is scaled by a guidance weight s and added to the diffusion model’s standard score prediction:

$$\hat{\epsilon}(x_t, t) = \epsilon_\theta(x_t, t) - \sqrt{1 - \bar{\alpha}_t} \cdot s \cdot \nabla_{x_t} \log p_\phi(y|x_t)$$

While highly effective for class-conditional generation, Classifier Guidance has critical flaws: it requires training a completely separate, noise-robust classifier for every new dataset, making it virtually impossible to scale to open-vocabulary T2I tasks where conditions are infinite.

3.4.2 Classifier-Free Guidance

To solve the scalability issues of the previous method, **Classifier-Free Guidance** (CFG) [34] was introduced. CFG completely eliminates the need for an auxiliary classifier model, elegantly allowing the core Diffusion model to guide itself. It has since become the absolute standard for nearly all T2I models. CFG operates by modifying the training phase slightly: the model is trained jointly on conditional data and unconditional data. This is achieved by randomly dropping the text prompt c (replacing it with an empty string $\emptyset$) for a small percentage of the training steps (e.g., 10%). Thus, the single network learns both $\epsilon_\theta(x_t, t, c)$ and $\epsilon_\theta(x_t, t, \emptyset)$.

During sampling, the model performs two forward passes at every single timestep t : one with the user’s prompt and one with a blank prompt. The final noise prediction extrapolates away from the unconditional prediction and toward the conditional one:

$$\hat{\epsilon}_t = \epsilon_\theta(x_t, t, \emptyset) + s \cdot (\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset))$$

Here, s is the **Guidance Scale**. If $s = 1$, it is standard conditional generation. If $s > 1$ (typically 7 to 10), the formula aggressively amplifies the features unique to the text prompt, forcing the model to strictly adhere to the user’s instructions and producing significantly more dynamic, high-fidelity images.

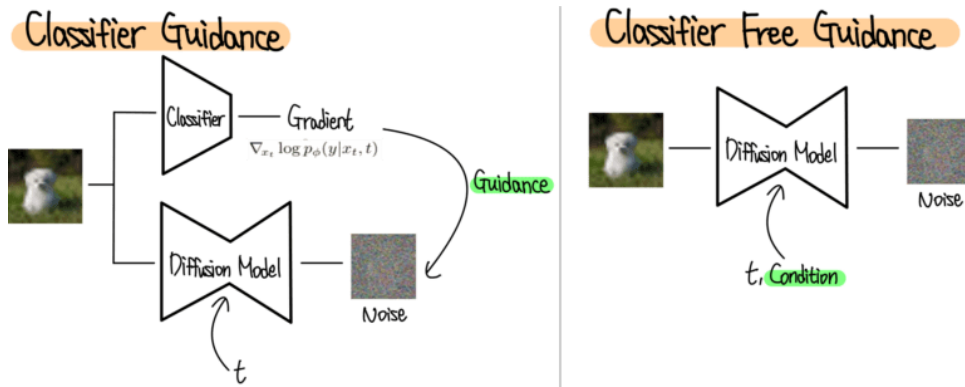


Figure 17: Classifier Guidance vs Classifier Free Guidance

In practical applications and consumer interfaces, the concept of the “unconditional” baseline $\emptyset$ has been ingeniously expanded. Instead of using a strictly empty string, users can supply a **Negative Prompt** (c_{neg}) alongside their primary **Positive Prompt** (c_{pos}).

The CFG equation is naturally adapted to accommodate this dual-conditioning:

$$\hat{\epsilon}_t = \epsilon_\theta(x_t, t, c_{neg}) + s \cdot (\epsilon_\theta(x_t, t, c_{pos}) - \epsilon_\theta(x_t, t, c_{neg}))$$

Mathematically, the network still computes two forward passes. However, instead of pushing the generation away from a generic, unconditioned prior, the formula explicitly pushes the trajectory *away* from the specific semantic features encoded in the negative prompt (e.g., “blurry, low resolution, mutated hands”) and *towards* the features of the positive prompt. This extrapolation acts as a powerful repulsive force against undesirable artifacts or concepts, granting users unprecedented, fine-grained control over the synthesis without altering the model’s weights.

3.4.3 Guidance Distillation

While CFG delivers exceptional prompt adherence, it introduces a severe computational bottleneck: it requires two full network evaluations per timestep. If a model takes 50 steps to generate an image, CFG requires 100 forward passes, effectively cutting the sampling speed in half. To overcome this, **Guidance Distillation** [35] was introduced. It leverages a Teacher-Student knowledge distillation framework to compress the CFG process into a single step.

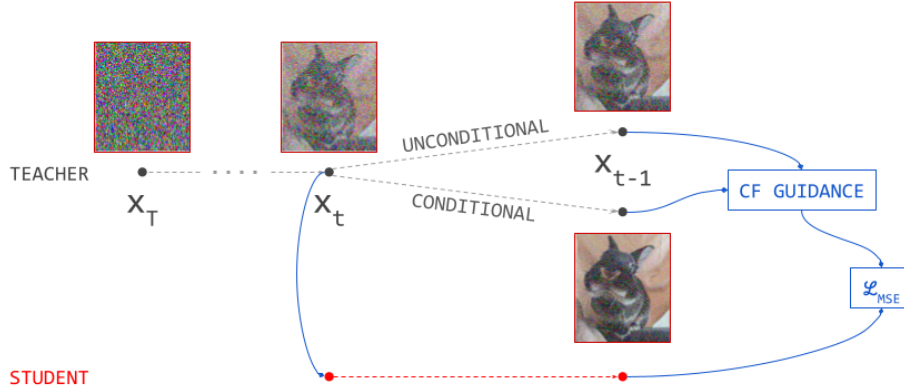


Figure 18: Guidance Distillation

The “Teacher” is the standard diffusion model running the full, expensive CFG algorithm (evaluating both conditional and unconditional paths to produce the combined target $\hat{\epsilon}_t$). The “Student” is a structurally identical but newly initialized model. The Student is trained to directly predict the Teacher’s final extrapolated output $\hat{\epsilon}_t$ using only a *single* forward pass:

$$\mathcal{L}_{\text{distill}} = \mathbb{E}_{x_t, c, t} \left[\left\| \epsilon_{\text{student}}(x_t, t, c) - \hat{\epsilon}_{\text{teacher_CFG}}(x_t, t, c, s) \right\|^2 \right]$$

By baking the CFG logic directly into the Student’s weights for a specific guidance scale s , Guidance Distillation successfully recovers the $2\times$ speed penalty, allowing for rapid, highly-aligned generation without the overhead of dual forward passes.

4 Adaptation Modules

The diffusion models are trained on billions of images using thousands of GPUs. Because their parameters encapsulate a vast, generalized understanding of the visual world, completely fine-tuning these models for specific tasks, custom art styles, or precise spatial control is computationally prohibitive and prone to catastrophic forgetting.

To harness the power of these frozen foundation backbones without altering their core knowledge, the generative AI community has developed a suite of **Adaptation Modules** (often broadly categorized under Parameter-Efficient Fine-Tuning, or PEFT). These modules are lightweight neural networks or mathematical interventions designed to “plug into” the pre-trained backbone, guiding the generation process toward specific constraints. Depending on how they interact with the core architecture (U-Net or DiT), these adapters can be rigorously classified into three distinct paradigms:

- **Weight Injection:** Methods that directly and efficiently modify the frozen weights of the backbone’s linear or convolutional layers to embed new concepts or artistic styles.
- **Feature Injection:** Methods that introduce external spatial structures (such as depth maps, poses, or inpainting masks) by computing parallel feature maps and adding them directly into the backbone’s residual streams.
- **Attention Injection:** Methods that intervene within the Cross-Attention blocks, decoupling text and visual embeddings to allow the model to natively attend to reference images.

4.1 Weight Injection Adapters

4.1.1 Low-Rank Adaptation

Originally introduced by Hu et al. specifically for Large Language Models (LLMs), **Low-Rank Adaptation** (LoRA) [36] was successfully ported to Diffusion Models in late 2022. It has since become the absolute standard for consumer-grade model fine-tuning, allowing users to train high-quality adapters on standard desktop GPUs.

The core principle of LoRA relies on the “intrinsic rank” hypothesis. Instead of updating the entire pre-trained weight matrix W , LoRA learns the change in weights (ΔW) by decomposing it into two smaller matrices. Consider a standard linear layer in a neural network (such as the projection layers W_Q, W_K, W_V within a Cross-Attention block) with a frozen pre-trained weight matrix $W \in \mathbb{R}^{m \times n}$. During normal inference, the output h given an input x is:

$$h = Wx$$

To fine-tune this layer, we mathematically want to learn an update ΔW , such that the new output is $h = (W + \Delta W)x$. LoRA constrains this update matrix ΔW by representing it as the product of two low-rank matrices, B and A :

$$h = Wx + \Delta Wx = Wx + (BA)x$$

where $A \in \mathbb{R}^{r \times n}$ and $B \in \mathbb{R}^{m \times r}$.

Crucially, r represents the rank of the matrix, which is chosen to be much smaller than the original dimensions ($r \ll m, n$). For example, if $m = 1024$ and $n = 1024$, full fine-tuning requires updating 1,048,576 parameters. If we set the rank $r = 16$, the matrices A and B only contain $(16 \times 1024) + (1024 \times 16) = 32,768$ parameters, reducing the trainable parameter count by over 96%.

During initialization, matrix A is populated with random Gaussian values, while matrix B is initialized to strictly zero. This ensures that at the very start of training, $\Delta W = BA = 0$, meaning the adapter acts as an identity function and does not disrupt the fragile pre-trained weights. Once trained, the adapter matrices B and A can be explicitly multiplied and permanently added back to W ($W' = W + BA$). This “weight merging” property is highly desirable as it results in zero additional inference latency.

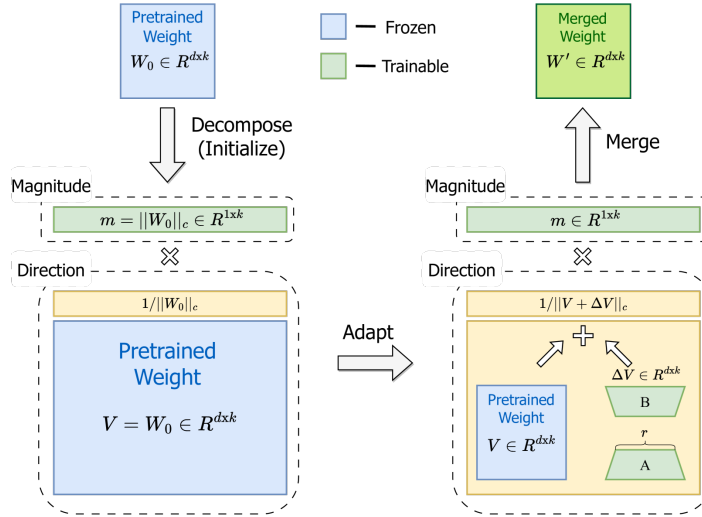


Figure 19: DoRA: Weight-Decomposed Low-Rank Adaptation

While standard LoRA is highly efficient, it inherently couples the magnitude and direction of the weight updates, sometimes struggling to capture proportional changes in feature magnitude compared to full fine-tuning. To address this limitation, **Weight-Decomposed Low-Rank Adaptation** (DoRA) [37] was introduced as a direct mathematical improvement. DoRA improves upon LoRA by applying a weight decomposition formula to separate the pre-trained weight matrix into magnitude and direction:

$$W = m \frac{V}{\|V\|}$$

where m is a learnable magnitude scalar and V is the directional matrix.

Instead of applying the standard LoRA directly to the entire weight, DoRA applies the low-rank updates (B and A) exclusively to the directional component V , while allowing the network to learn the magnitude m as a separate parameter. This subtle decoupling allows DoRA to achieve learning capacities much closer to full fine-tuning while maintaining the extreme parameter efficiency that made standard LoRA popular.

4.1.2 HyperNetworks

Before LoRA became the undisputed standard for parameter-efficient fine-tuning, the generative AI community heavily relied on **HyperNetworks** [38] to inject custom concepts, particularly specific artistic styles, into frozen diffusion models. Originally proposed by Ha et al. for recurrent neural networks, the concept was popularized in the diffusion space around late 2022 by communities aiming to stylize diffusion models without the massive hardware requirements of full fine-tuning. While LoRA learns a static, fixed matrix of weight offsets (ΔW), a HyperNetwork takes a fundamentally different, dynamic approach. A HyperNetwork is an auxiliary, secondary neural network whose explicit purpose is to generate the weights (or weight modulations) for another primary neural network (the denoising backbone).

In a standard neural network layer, the output h is computed using a static weight matrix W and input x : $h = Wx$. In a pure HyperNetwork architecture, the weight matrix W is not a static parameter but a dynamic output of a secondary network $\mathcal{H}$, parameterized by ϕ . This secondary network takes a condition c (which could be the text prompt, the timestep, or the input feature itself) and outputs the weights for the primary layer:

$$W = \mathcal{H}_\phi(c)$$

The primary layer then computes $h = \mathcal{H}_\phi(c)x$. This makes the primary network highly expressive, as its actual mathematical structure changes dynamically based on the conditioning signal.

Generating the full weight matrices for the massive convolutional or linear layers of a U-Net is computationally intractable. Therefore, the implementation of HyperNetworks in LDMs was significantly adapted to be computationally feasible. Instead of generating the weights for the entire model, HyperNetworks are implemented as tiny, independent MLPs that are physically attached to the Cross-Attention blocks of the frozen U-Net. Specifically, they intercept the intermediate feature tensors—most commonly the Keys ($\mathbf{K}$) and Values ($\mathbf{V}$) derived from the text encoder. Let $\mathbf{K} = \mathbf{C}\mathbf{W}_K$ and $\mathbf{V} = \mathbf{C}\mathbf{W}_V$ be the standard projections. The HyperNetwork, consisting of two linear layers separated by a non-linear activation (like ReLU or Mish) and potentially Dropout, applies a non-linear transformation to these vectors:

$$\begin{aligned}\mathbf{K}' &= \mathbf{K} + \mathcal{H}_{\phi_K}(\mathbf{K}) \\ \mathbf{V}' &= \mathbf{V} + \mathcal{H}_{\phi_V}(\mathbf{V})\end{aligned}$$

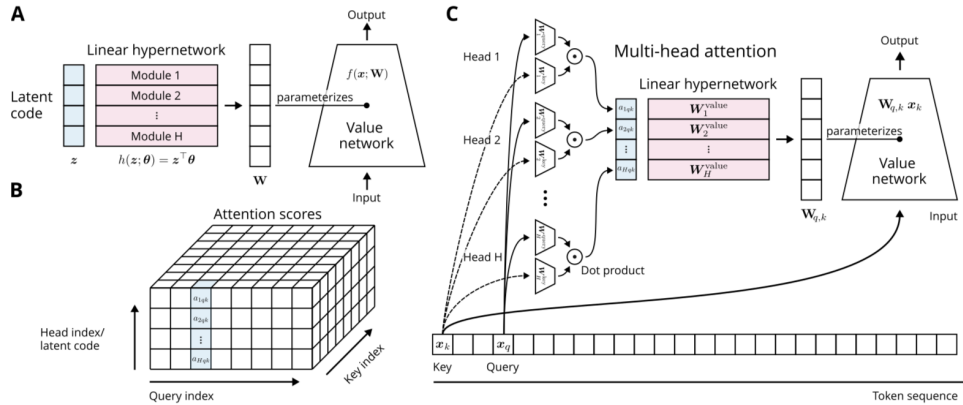


Figure 20: Attention as a Hypernetwork

These modulated keys and values ($\mathbf{K}'$, $\mathbf{V}'$) are then fed into the standard attention equation.

While theoretically more expressive due to this non-linearity, HyperNetworks suffer from significant drawbacks: they are notoriously difficult to train, prone to overfitting, sensitive to hyperparameter tuning (such as learning rates), and they add measurable latency during inference because their operations cannot be merged into the base weights. Consequently, while historically significant as an early adaptation mechanism, they have largely been superseded by the mathematical elegance and stability of LoRA.

4.1.3 HyperAlign

HyperAlign [39] represents a novel framework that bridges the dynamic expressiveness of HyperNetworks with the parameter efficiency of LoRA for efficient test-time alignment. It addresses the

difficult trade-off present in existing alignment methods: fine-tuning approaches often suffer from reward over-optimization and loss of diversity, while test-time scaling methods introduce significant computational overhead and tend to under-optimize. Instead of modifying latent states directly through gradient-based guidance, HyperAlign dynamically generates LoRA weights to modulate the generative model’s operators. Temporally integrating these generated LoRA weights into the original model parameters yields an input-and-step-specific score function parameterized by $\theta + \Delta\theta_t$. This allows the denoising trajectory to be adaptively adjusted for reward-conditioned alignment without the prohibitive cost of generating full model parameters.

To enhance inference efficiency, the framework introduces three distinct weight generation strategies:

- **HyperAlign-S (Step-wise):** Generates LoRA weights at all denoising steps for fine-grained alignment.
- **HyperAlign-I (Initial):** Generates weights only once at the starting point, applying the same adapter for minimal computation.
- **HyperAlign-P (Piece-wise):** Updates adapters only at dynamically identified key timesteps, balancing generation performance and computational efficiency.

During training, the hypernetwork is optimized using a reward score objective that is regularized with preference data. This regularization prevents the model from overfitting to proxy scores (a phenomenon known as reward hacking) while maintaining fidelity to genuine human preferences and preserving the generative diversity of the base model.

4.2 Feature Injection Adapters

While Weight Injection methods excel at altering the global style or semantic knowledge of a diffusion model, they lack the spatial precision required to control exact compositions, poses, or structural layouts. Feature Injection Adapters solve this by computing parallel spatial feature maps based on structural conditions and explicitly injecting (adding) them into the intermediate activation layers of the frozen denoising backbone.

4.2.1 ControlNet

Introduced by Zhang and Agrawala, **ControlNet** [40] revolutionized the controllability of T2I diffusion models. It addressed a critical limitation: how can we condition a pre-trained model on dense, pixel-level spatial structures (such as edges or human poses) without catastrophically forgetting the model’s original generative capabilities and without requiring massive hardware for retraining?

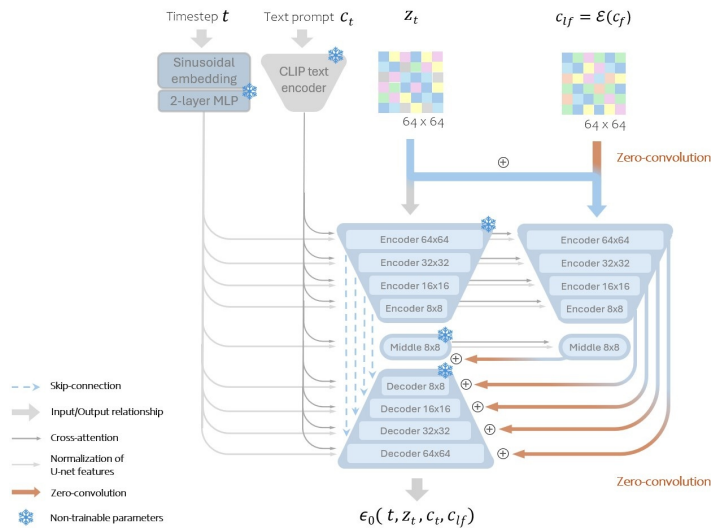


Figure 21: ControlNet applied to Diffusion Model

ControlNet achieves absolute structural control over the generated image by establishing a parallel network branch that extracts multi-scale features from the conditioning image and injects them directly into the decoder of the frozen base U-Net. The mathematical ingenuity of ControlNet lies in its use of **Zero Convolutions**. A Zero Convolution is defined as a 1×1 convolutional layer where both the weights and biases are explicitly initialized to zero.

Let $\mathcal{F}(x; \Theta)$ represent a neural network block (e.g., a ResNet block in the U-Net) with frozen pre-trained parameters Θ , taking an input feature map x . ControlNet creates a trainable copy of this block, parameterized by Θ_c , alongside two Zero Convolution layers, $\mathcal{Z}_1$ and $\mathcal{Z}_2$. Given an external conditioning signal c , the modified forward pass is formulated as follows:

$$y_c = \mathcal{F}(x; \Theta) + \mathcal{Z}_2\left(\mathcal{F}_{copy}(x + \mathcal{Z}_1(c; \Theta_{z1}); \Theta_c); \Theta_{z2}\right)$$

Because the parameters Θ_{z1} and Θ_{z2} are initialized to zero, at the very first step of training, $\mathcal{Z}_1(c) = 0$ and $\mathcal{Z}_2(\dots) = 0$. Consequently, $y_c = \mathcal{F}(x; \Theta)$. This mathematical guarantee ensures that the untrained adapter behaves exactly like an identity function, entirely preserving the pre-trained T2I prior. As training progresses, the gradients flow through the zero convolutions, gradually learning to map the spatial conditions into the generative feature space.

A common misconception is that ControlNet’s input condition c must be a traditional “image”. In reality, the architecture is entirely agnostic to the semantic nature of the spatial tensor. ControlNet can process a vast array of practical input modalities:

- **Geometric Data:** Canny edge maps, HED boundaries, or Depth maps (providing 3D spatial awareness).
- **Abstract Data:** Semantic segmentation maps (defining regional classes) or OpenPose skeletons (defining exact human biomechanical poses).
- **Non-Visual Data:** ControlNet can adapt to data that is not inherently optical. For instance, **Audio** can be injected by converting sound files into Spectrograms (visual graphs of frequency over time). Furthermore, **Text and Typography** can be processed as binary spatial masks, allowing the diffusion model to render highly stylized, legible text within the generated scene.

4.2.2 BrushNet

While ControlNet is exceptional for holistic spatial guidance, it struggles with the specific task of **Image Inpainting** (modifying specific masked regions of an image while strictly preserving the unmasked background). If standard ControlNet or naive feature concatenation is used for inpainting, the model often blends the masked and unmasked regions unnaturally, or the spatial condition interferes with the text prompt, leading to a degradation of the base model’s T2I semantic prior.

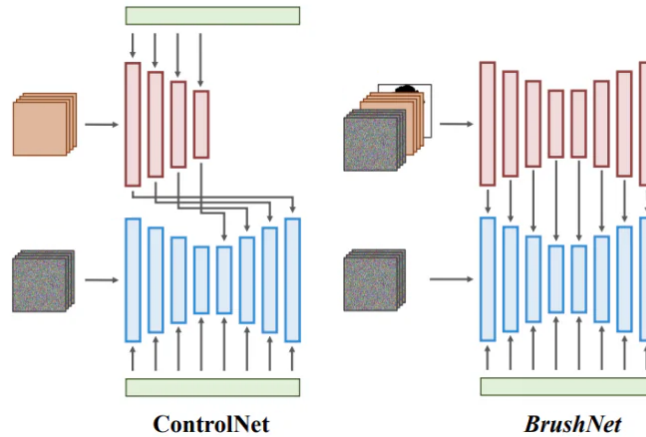


Figure 22: Comparison of ControlNet and BrushNet

BrushNet [41] was introduced as a specialized, plug-and-play feature injection adapter explicitly designed to solve the pixel-level charge of image inpainting. The core philosophy of BrushNet is the

strict decoupling of tasks. The frozen pre-trained U-Net is entrusted solely with understanding the text prompt and synthesizing semantic content, while the parallel BrushNet branch is responsible exclusively for processing the masked image features and ensuring seamless boundary blending.

To achieve this, BrushNet employs a modified U-Net encoder architecture. The input to the BrushNet branch is a concatenation of the masked image I_{masked} (the known pixels) and the binary mask M (indicating the regions to be generated). Crucially, unlike ControlNet, BrushNet **removes all Cross-Attention layers** from its architecture. By intentionally blinding the adapter to the text prompt, BrushNet is forced to focus entirely on extracting low-level structural and color features from the unmasked background. During the forward pass, the BrushNet branch extracts a set of hierarchical spatial features $\mathbf{H}_B^i$ at various resolutions. These features are then injected into the corresponding layers of the frozen pre-trained U-Net ($\mathbf{H}_U^i$) through a zero-initialized convolutional block, often modulated by a controllable weight scale w :

$$\mathbf{H}_{out}^i = \mathbf{H}_U^i + w \cdot \mathcal{Z}(\mathbf{H}_B^i)$$

This decoupled feature injection guaranties that the model strictly preserves the unmasked pixels (guided heavily by $\mathbf{H}_B^i$) while naturally hallucinating the missing content inside the mask using the pure, uncorrupted semantic power of the base T2I model ($\mathbf{H}_U^i$).

4.3 Attention Injection Adapters

While Weight Injection alters the internal parameters of a model and Feature Injection adds external spatial signals, **Attention Injection** operates directly within the Cross-Attention blocks of the denoising backbone. This paradigm focuses on how the model maps external conditional modalities (such as reference images) into the sequence-to-sequence attention mechanism without overwriting the established T2I semantic priors.

4.3.1 IP-Adapter

In standard T2I generation, conveying complex visual concepts, specific artistic styles, or precise character identities through text prompts alone is often arduous and imprecise. The **Image Prompt Adapter** (IP-Adapter) [42], introduced by Ye et al., solves this by enabling pre-trained diffusion models to accept visual reference images directly as prompts alongside text. Unlike ControlNet, which injects dense spatial maps to control structure, IP-Adapter extracts the global semantic essence of an image and injects it into the cross-attention layers, functioning exactly like a visual text prompt.

To process an image prompt, IP-Adapter utilizes a pre-trained CLIP Image Encoder to extract global image embeddings. However, diffusion models expect a sequence of tokens (like words in a sentence), not a single global vector. Therefore, IP-Adapter employs a lightweight trainable linear projection network (often implemented as a MLP or a Perceiver Resampler) to map the global CLIP image features into a sequence of image tokens, denoted as $\mathbf{C}_{img} \in \mathbb{R}^{L_{img} \times d}$.

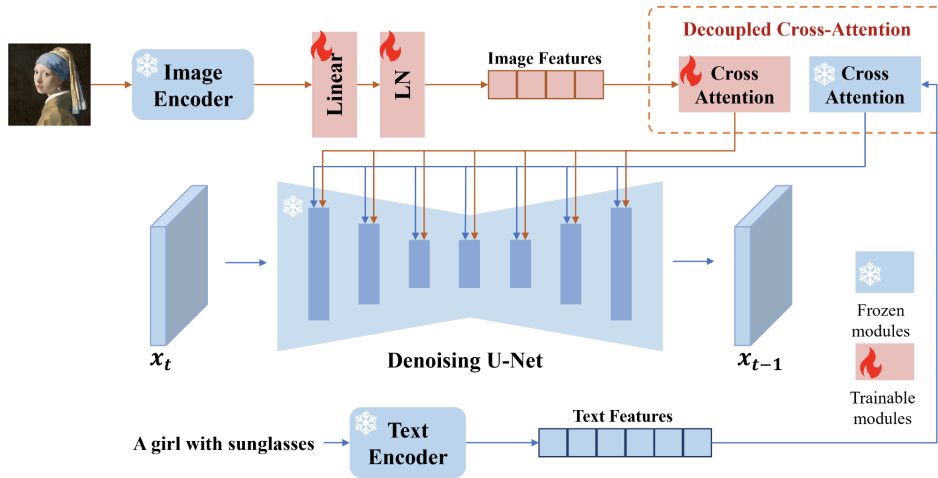


Figure 23: IP-Adapter with decoupled cross-attention strategy

The core architectural innovation of IP-Adapter is its **Decoupled Cross-Attention** mechanism. A naive approach to integrating image tokens would be to simply concatenate them with the text tokens ($\mathbf{C}_{text}$) into a single long sequence and pass them through the pre-trained Cross-Attention block. However, this forces the pre-trained Softmax function to suddenly re-learn how to distribute attention across completely new modalities, often severely degrading the model’s text-adherence capabilities. Instead, IP-Adapter creates a completely separate, parallel cross-attention pathway exclusively for the image tokens. Let $\mathbf{Z}$ be the spatial feature queries of the U-Net. The pre-trained text cross-attention is computed using frozen weights $\mathbf{W}_K$ and $\mathbf{W}_V$:

$$\begin{aligned}\mathbf{Q} &= \mathbf{Z}\mathbf{W}_Q \\ \mathbf{K}_{text} &= \mathbf{C}_{text}\mathbf{W}_K \\ \mathbf{V}_{text} &= \mathbf{C}_{text}\mathbf{W}_V \\ \text{Attn}_{text} &= \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}_{text}^T}{\sqrt{d}}\right)\mathbf{V}_{text}\end{aligned}$$

IP-Adapter introduces new, trainable weight matrices $\mathbf{W}'_K$ and $\mathbf{W}'_V$ specifically for the image tokens:

$$\begin{aligned}\mathbf{K}_{img} &= \mathbf{C}_{img}\mathbf{W}'_K, \quad \mathbf{V}_{img} = \mathbf{C}_{img}\mathbf{W}'_V \\ \text{Attn}_{img} &= \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}_{img}^T}{\sqrt{d}}\right)\mathbf{V}_{img}\end{aligned}$$

The final output of the modified attention block is the linear summation of both attention outputs, modulated by a weight factor λ :

$$\mathbf{Z}_{out} = \text{Attn}_{text} + \lambda \cdot \text{Attn}_{img}$$

This decoupled architecture provides three fundamental mathematical and practical advantages:

- **Preservation of Original Knowledge:** The text attention pathway (Attn_{text}) remains mathematically isolated and is not disrupted by the image data. The original T2I prior is perfectly preserved, and the image functions purely as a supplementary signal.
- **Fine-tuning Efficiency:** During training, the base U-Net is completely frozen. The model only needs to optimize the lightweight image projection network and the new attention matrices ($\mathbf{W}'_K$ and $\mathbf{W}'_V$). This converges significantly faster than concatenating tokens because the network does not have to explicitly “re-teach” the original Softmax operation how to balance the attention distribution between intertwined text and image tokens.
- **Flexibility and Control:** Because the outputs of the two attention mechanisms are combined via a simple linear addition, users can effortlessly adjust the influence of the image prompt dynamically during inference by altering the scalar λ . If the tokens were concatenated prior to the Softmax function, isolating and scaling the influence of the image independently of the text would be mathematically highly complex and unstable.

5 Open Source Models

The landscape of Conditional Diffusion Models for T2I generation has evolved into two distinct paradigms based on their accessibility and distribution: **Closed Source** (proprietary) and **Open Source** (open-weights) models.

The **Closed Source** branch encompasses commercial, API-gated systems developed by large technology corporations. Prominent examples include Midjourney, OpenAI’s DALL-E series, and Google’s Imagen. While these models often provide highly polished, consumer-ready out-of-the-box, their underlying architectures, training methodologies, and network weights remain strictly confidential. This opacity prevents the broader research community from deeply analyzing, modifying, or locally fine-tuning them for specialized tasks.

Conversely, the **Open Source** branch has been the primary catalyst for the rapid democratization and academic advancement of generative AI. By publicly releasing the model weights, creators enable researchers and developers to run these models locally, inspect their internal mechanics, and develop the

robust ecosystem of adaptation modules discussed in Chapter 4 (such as LoRA, ControlNet, and IP-Adapter). Key representatives of this open-weights philosophy include Stable Diffusion, FLUX, Tencent’s Hunyuan-DiT, and Kandinsky.

This chapter examines the architectural realizations of these SOTA open-source foundation models. While we acknowledge the contributions of the broader ecosystem, including Hunyuan-DiT and Kandinsky, the primary focus of the following sections will be centered on the two most transformative lineages in the field: **Stable Diffusion**, which pioneered and dominated the Convolutional U-Net era of Latent Diffusion Models, and **FLUX**, which represents the current pinnacle of the Transformer and Flow-Matching era.

5.1 Stable Diffusion

The Stable Diffusion [3] family, originally born from a collaboration between the CompVis group at LMU Munich, Runway, and Stability AI, represents the most historically significant lineage of open-source generative AI. Its release fundamentally democratized high-resolution image synthesis, shifting the power from proprietary corporate APIs to consumer-grade hardware. Over its iterations, the architecture has undergone radical transformations, evolving from standard Convolutional U-Nets to advanced Multimodal Transformers.

5.1.1 Stable Diffusion 1.x

The public release of **Stable Diffusion 1.4** in August 2022 marked a watershed moment in the AI community. Built upon the LDM framework, it was designed to run efficiently on standard desktop GPUs with as little as 8GB of VRAM.

- **Perceptual Compression:** It utilized a VAE with a downsampling factor of $f = 8$. This efficiently compressed standard 512×512 pixel images into a highly manageable 64×64 latent space.
- **Conditioning:** For text comprehension, the model relied on OpenAI’s pre-trained **CLIP ViT-L/14** text encoder.
- **Backbone:** The core denoising engine was a Convolutional U-Net containing approximately 860 million parameters.

In October 2022, Runway released **Stable Diffusion 1.5**. Architecturally identical to 1.4, SD 1.5 was trained for an additional 2.5 times longer. Interestingly, this release was steeped in controversy: Runway independently published the SD 1.5 weights to Hugging Face against the wishes of Stability AI, sparking a brief but intense corporate dispute over intellectual property and release safety. Despite the drama, the extended training granted the model superior aesthetic coherence. SD 1.5 quickly became the absolute gold standard and the most widely fine-tuned foundation model in the open-source community for years to come.

5.1.2 Stable Diffusion 2.x

Released shortly after in late 2022 (November for SD 2.0 and December for SD 2.1), the 2.x series attempted to improve the fundamental mathematical and semantic capabilities of the model.

- **Text Encoder Shift:** Stability AI deliberately moved away from OpenAI’s proprietary CLIP encoder, opting instead for the fully open-source **OpenCLIP ViT-H/14**. While theoretically more robust, this change drastically altered how the model interpreted prompts, breaking backward compatibility with the massive library of community “prompt engineering” techniques developed for SD 1.5.
- **Velocity Prediction Objective:** Instead of predicting the added noise (ϵ -prediction), SD 2.x was trained to predict the velocity (v -prediction) of the diffusion trajectory. This mathematical shift significantly improved the model’s ability to generate extreme contrasts (very dark or very bright images) and facilitated native training at higher resolutions (768×768).

Despite its technical merits, the 2.x series was notoriously poorly received. Aggressive filtering of the training dataset to remove copyrighted artists and NSFW content inadvertently lobotomized the model’s anatomical understanding and artistic flexibility. Consequently, the open-source community largely boycotted the 2.x series, preferring to continue building on the more expressive SD 1.5 ecosystem.

5.1.3 Stable Diffusion XL

Introduced in July 2023, **Stable Diffusion XL** (SDXL) represented the ultimate, maximalist push to exploit the traditional Convolutional U-Net architecture before the industry shifted to Transformers.

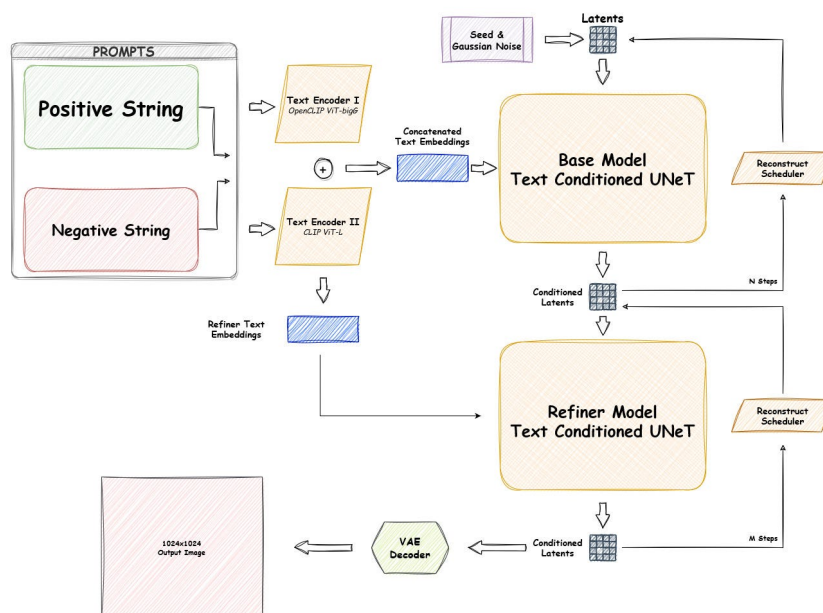


Figure 24: SDXL architecture

SDXL was a massive leap in scale and complexity. It featured a natively trained resolution of 1024×1024 and aggressively expanded the parameter count of the U-Net backbone to 2.6 billion parameters. To handle complex prompts, it utilized a dual-encoder conditioning mechanism, concatenating the embeddings from OpenCLIP ViT-bigG and CLIP ViT-L. Furthermore, SDXL introduced an optional, secondary “Refiner” model (an 0.9 billion parameter U-Net) specifically trained on low-noise levels to enhance high-frequency details in the final steps of generation. However, in practical community workflows, the Refiner was quickly abandoned. Users found that utilizing the Base model alone, especially after community fine-tuning, achieved superior results without the doubled VRAM requirements and inference latency of a two-stage pipeline.

5.1.4 Stable Diffusion 3.x

By March 2024, the limitations of the U-Net architecture had become apparent. A critical flaw in SDXL and its predecessors was **Attribute Bleeding**. Because text processing in U-Nets is isolated and only injected globally via standard Cross-Attention, the model often conflated positional attributes. For example, given the prompt “a red shirt and blue pants”, the model might incorrectly generate a blue shirt and red pants. **Stable Diffusion 3 (SD3)** addressed this through a complete architectural overhaul:

- **Mathematical Foundation:** SD3 abandoned standard diffusion formulations in favor of **Rectified Flow**, creating straighter, more efficient generation trajectories.
- **Triple Text Encoders:** To completely solve prompt adherence, SD3 simultaneously employed three massive text encoders: OpenCLIP-ViT/G, CLIP-ViT/L, and crucially, the pure-language **T5-xxl** model.
- **Multimodal Diffusion Transformer (MMDiT):** The U-Net was entirely replaced by a Transformer backbone. To solve Attribute Bleeding, MMDiT introduced a **Joint Attention** mechanism. Instead of the text embeddings remaining frozen and merely acting as a reference for the image, Joint Attention concatenates the text and image tokens, allowing them to mutually attend to each other. The text representations are dynamically updated and refined alongside the image features through every layer of the network, ensuring precise spatial binding of attributes.

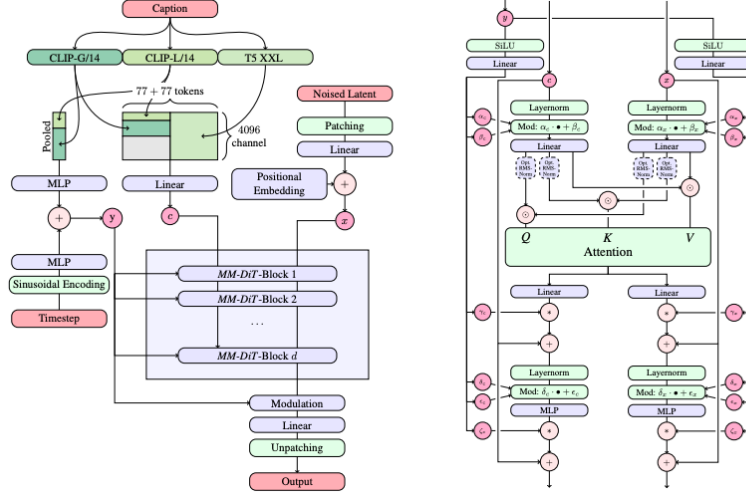


Figure 25: Multimodal Diffusion Transformer architecture

In October 2024, following community feedback regarding early SD3 models struggling with human anatomy and restrictive alignment, Stability AI released **Stable Diffusion 3.5**. This iteration relaxed the overly aggressive alignment strategies, rebalanced the training data to restore anatomical diversity, and introduced architectural refinements such as **Query-Key Normalization** within the transformer blocks to stabilize training dynamics and improve feature representation across modalities.

5.2 FLUX

Following the turbulent launch and mixed reception of Stable Diffusion 3, the open-source community witnessed a dramatic paradigm shift led by a newly formed research group: **Black Forest Labs**. Founded in early 2024 by Robin Rombach, Andreas Blattmann, and Patrick Esser—the original lead researchers behind Stable Diffusion during their tenure at Stability AI and LMU Munich—BFL was established with a clear mandate: to perfect the Flow Matching and Multimodal Transformer architectures that previous models had struggled to stabilize.

The culmination of this effort is the **FLUX** [43] series, which rapidly established itself as the new definitive standard for both open-weight and commercial T2I generation, effectively challenging proprietary giants like Midjourney and DALL-E 3. Unlike traditional diffusion models that predict Gaussian noise, the entire FLUX lineage is built upon **Flow Matching** (specifically Rectified Flow), which enforces deterministic, straight-line trajectories between noise and data distributions for highly efficient sampling. Architecturally, FLUX employs a **Hybrid Diffusion Transformer (HybridDiT)**.

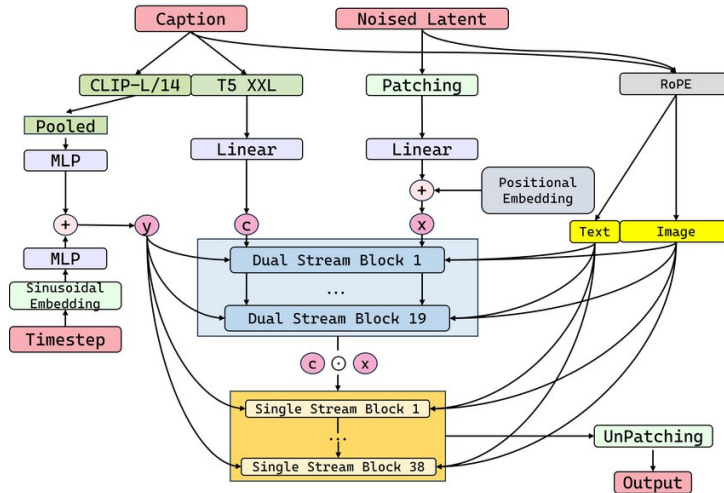


Figure 26: FLUX architecture

Rather than using a pure joint-attention mechanism or entirely separated streams, the HybridDiT architecture elegantly weaves together parallel attention blocks and multimodal cross-attention blocks. It processes a massive sequence of tokens derived from both the latent image patches and dual text encoders (**CLIP-ViT/L** and the massive pure-language **T5-xxl**), allowing for unprecedented semantic comprehension and complex spatial reasoning.

Released in August 2024, the first generation of FLUX was a monumental leap in generative AI, setting a new benchmark for prompt adherence, anatomical consistency, and especially *text rendering* (typography) within images.

- **Scale:** The base model featured a massive **12 billion parameters**, making it significantly larger than any preceding open-source model (compared to SDXL’s 2.6B and SD3’s 8B).
- **Ecosystem:** BFL released the model in three tiers: **FLUX.1 [pro]** (closed-source, API only), **FLUX.1 [dev]** (open-weights with a non-commercial license for researchers), and **FLUX.1 [schnell]** (a highly distilled, 4-step model released under the permissive Apache 2.0 license).

The release of FLUX.1 caused a massive migration of the open-source community. Creators abandoned the fractured SDXL and SD3 ecosystems to build fine-tunes, LoRAs, and ControlNets exclusively for FLUX.1, praising its “Midjourney-level” out-of-the-box aesthetics. This dominance was further cemented in October 2024 with the release of **FLUX 1.1 [pro]**, offering a 6x speedup and improved compositional diversity.

On November 25, 2025, Black Forest Labs disrupted the industry once again with the highly anticipated release of the **FLUX.2** family. If FLUX.1 was about perfecting T2I quality, FLUX.2 was engineered to solve professional, production-grade workflows and absolute temporal consistency. The FLUX.2 architecture pushed the absolute boundaries of hardware constraints, scaling up to an astonishing **32 billion parameters** for its flagship [dev] and [pro] models. At full precision, the model required nearly 90GB of VRAM, pushing the community to heavily adopt FP8 quantization techniques (in collaboration with NVIDIA) to fit the massive model onto 24GB consumer GPUs like the RTX 4090. Beyond raw parameter scaling, FLUX.2 introduced several transformative features:

- **Unified Generation and Editing:** Unlike previous generations that required separate checkpoints for inpainting or I2I tasks, FLUX.2 natively fused T2I, single-reference editing, and complex semantic modification into a single, unified architectural pipeline.
- **Multi-Reference Support:** The most praised feature by the creative community was native multi-referencing. FLUX.2 allowed users to input up to 10 reference images simultaneously, achieving flawless character, style, and product consistency without the need to train time-consuming LoRAs.
- **Grounded Generation:** FLUX.2 introduced web-grounding capabilities, allowing the model to incorporate real-time context and world knowledge to faithfully render recent concepts and highly accurate structural details.
- **The [klein] Variant:** Acknowledging the extreme hardware demands of the 32B model, BFL also released **FLUX.2 [klein]**, a dense 4B parameter model running in under a second on standard 13GB VRAM hardware (under Apache 2.0 license), cementing their dominance across all tiers of hardware accessibility.

5.3 Hunyuan-DiT

While Western companies like Stability AI and Black Forest Labs dominated the early narrative of open-source generative AI, the landscape shifted significantly in early 2024 with the entry of **Hunyuan-DiT** [44], developed by the Chinese technology giant Tencent. Hunyuan-DiT adopted the Diffusion Transformer (DiT) architecture early on, parallel to the development of SD3 and FLUX. However, it introduced several distinct engineering choices:

- **Rotary Positional Embeddings (RoPE):** Instead of the standard sinusoidal positional encodings used in baseline DiTs, Hunyuan-DiT integrated RoPE (a technique borrowed from advanced Large Language Models like LLaMA). This allowed the network to better extrapolate spatial relationships and coordinate complex multi-object compositions.

- **Native Bilingual Capability:** Recognizing the heavy English-centric bias in models like Stable Diffusion, Tencent trained Hunyuan-DiT with a robust dual-language text encoder pipeline. It seamlessly understands both English and Chinese, demonstrating an unprecedented capability to render Chinese typography and deeply culturally specific elements (such as traditional Chinese poetry, historical attire, and idioms) that Western models consistently fail to generate.

The open-source release of Hunyuan-DiT was highly unexpected by the global community, as Chinese megacorporations traditionally kept their most advanced AI models proprietary. During the period when the community was highly critical of Stable Diffusion 3’s anatomical failures, Hunyuan-DiT served as a highly stable, structurally sound alternative, proving that the DiT architecture could be successfully scaled and deployed in a fully open-weights ecosystem.

5.4 Kandinsky

The **Kandinsky** [45] series, developed by the AI-Forever team (backed by Russian researchers at Sber AI), represents one of the most persistent and fascinating “underdog” lineages in the open-source T2I ecosystem. While it never achieved the absolute mainstream dominance of Stable Diffusion, it maintained a fiercely loyal cult following among digital artists.

While Stable Diffusion popularized the standard LDM approach using cross-attention, Kandinsky 2.1 and 2.2 took heavy inspiration from OpenAI’s DALL-E 2. It utilized an **unCLIP (Prior-based)** architecture. Instead of feeding text embeddings directly into the denoising U-Net, Kandinsky employed a two-stage process:

- **The Prior Model:** A separate diffusion model first maps the CLIP text embedding into a CLIP image embedding.
- **The Decoder:** The main U-Net then takes this synthesized image embedding and generates the actual pixels (or latents).

This architectural quirk made Kandinsky 2.x incredibly powerful for tasks like image mixing and style transfer, as the model inherently operates in a unified text-image semantic space.

With the release of **Kandinsky 3.0** in late 2023, the team abandoned the prior-based unCLIP architecture in favor of a more traditional, massive U-Net structure combined with the FLAN-UL2 large language model for text encoding. Within the community, Kandinsky is uniquely celebrated for its default aesthetic. While Stable Diffusion models often require complex prompt engineering, negative prompts, and LoRAs to achieve a stylized look, Kandinsky is heavily curated to produce highly artistic, painterly, and cinematic outputs almost effortlessly. It natively understands Russian text and folklore, offering a distinct cultural counterweight to the heavily Western-biased datasets of its competitors.

References

- [1] Midjourney. *Midjourney Documentation*. <https://docs.midjourney.com/>
- [2] Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). *Hierarchical Text-Conditional Image Generation with CLIP Latents* (DALL-E 2). <https://arxiv.org/abs/2204.06125>
- [3] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/2112.10752>
- [4] Kingma, D. P., & Welling, M. (2013). *Auto-Encoding Variational Bayes*. <https://arxiv.org/abs/1312.6114>
- [5] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). *Generative Adversarial Nets*. Advances in Neural Information Processing Systems, 27. <https://arxiv.org/abs/1406.2661>
- [6] Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015). *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. International Conference on Machine Learning (ICML). <https://arxiv.org/abs/1503.03585>
- [7] Liu, X., Gong, C., & Liu, Q. (2023). *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2209.03003>
- [8] Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow Matching for Generative Modeling*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2210.02747>
- [9] Song, Y., & Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. Advances in Neural Information Processing Systems, 32. <https://arxiv.org/abs/1907.05600>
- [10] Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. Advances in Neural Information Processing Systems, 33. <https://arxiv.org/abs/2006.11239>
- [11] Kullback, S., & Leibler, R. A. (1951). *On Information and Sufficiency*. The Annals of Mathematical Statistics, 22(1), 79–86. <https://doi.org/10.1214/aoms/1177729694>
- [12] Song, J., Meng, C., & Ermon, S. (2020). *Denoising Diffusion Implicit Models*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2010.02502>
- [13] Salimans, T., & Ho, J. (2022). *Progressive Distillation for Fast Sampling of Diffusion Models*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2202.00512>
- [14] Liu, X., Zhang, X., Ma, J., Peng, J., & Liu, Q. (2024). *InstaFlow: One Step is Enough for High-Quality Diffusion-Based Text-to-Image Generation*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2309.06380>
- [15] Yan, H., Liu, X., Pan, J., Liew, J. H., Liu, Q., & Feng, J. (2024). *PeRFlow: Piecewise Rectified Flow for Text-to-Image Generation*. Conference on Neural Information Processing Systems (NeurIPS). <https://piecewise-rectified-flow.github.io/>
- [16] Yoon, J., & Lee, J. (2024). *Sequential Flow Straightening for Generative Modeling*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2402.06461>
- [17] Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. Medical Image Computing and Computer-Assisted Intervention (MICCAI). <https://arxiv.org/abs/1505.04597>
- [18] Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). *Image-to-Image Translation with Conditional Adversarial Networks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1611.07004>

- [19] Salimans, T., Karpathy, A., Chen, X., & Kingma, D. P. (2017). *PixelCNN++: Improving the Pixel-CNN with Discretized Logistic Mixture Likelihood and Other Modifications*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1701.05517>
- [20] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/1512.03385>
- [21] Wu, Y., & He, K. (2018). *Group Normalization*. Proceedings of the European Conference on Computer Vision (ECCV). <https://arxiv.org/abs/1803.08494>
- [22] Elfving, S., Uchibe, E., & Doya, K. (2018). *Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning*. <https://arxiv.org/abs/1702.03118>
- [23] Peebles, W., & Xie, S. (2023). *Scalable Diffusion Models with Transformers*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). <https://arxiv.org/abs/2212.09748>
- [24] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2010.11929>
- [25] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. International Conference on Machine Learning (ICML). <https://arxiv.org/abs/2103.00020>
- [26] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research (JMLR). <https://arxiv.org/abs/1910.10683>
- [27] Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL). <https://arxiv.org/abs/1508.07909>
- [28] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/1706.03762>
- [29] Misra, D. (2020). *Mish: A Self Regularized Non-Monotonic Activation Function*. British Machine Vision Conference (BMVC). <https://arxiv.org/abs/1908.08681>
- [30] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization*. <https://arxiv.org/abs/1607.06450>
- [31] Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E., ... & Norouzi, M. (2022). *Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding*. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2205.11487>
- [32] DeepFloyd Lab. (2023). *DeepFloyd IF: A Novel State-of-the-Art Open-Source Text-to-Image Model*. <https://github.com/deep-floyd/IF>
- [33] Dhariwal, P., & Nichol, A. (2021). *Diffusion Models Beat GANs on Image Synthesis*. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2105.05233>
- [34] Ho, J., & Salimans, T. (2022). *Classifier-Free Diffusion Guidance*. NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications. <https://arxiv.org/abs/2207.12598>
- [35] Meng, C., Rombach, R., Gao, R., Kingma, D. P., Ermon, S., Ho, J., & Salimans, T. (2023). *On Distillation of Guided Diffusion Models*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://arxiv.org/abs/2210.03142>
- [36] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2106.09685>

- [37] Liu, S. Y., Wang, C. Y., Yin, H., Molchanov, P., Wang, Y., Cheng, K. T., & Chen, M. H. (2024). *DoRA: Weight-Decomposed Low-Rank Adaptation*. International Conference on Machine Learning (ICML). <https://arxiv.org/abs/2402.09353>
- [38] Ha, D., Dai, A., & Le, Q. V. (2016). *Hypernetworks*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1609.09106>
- [39] Xie, X., Guo, J., & Gong, D. (2026). *HyperAlign: Hypernetwork for Efficient Test-Time Alignment of Diffusion Models*. <https://arxiv.org/abs/2601.15968>
- [40] Zhang, L., Rao, A., & Agrawala, M. (2023). *Adding Conditional Control to Text-to-Image Diffusion Models*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). <https://arxiv.org/abs/2302.05543>
- [41] Ju, X., Li, X., Zhang, B., Jin, Y., Lin, M., Zhao, T., ... & Zheng, Y. (2024). *BrushNet: A Plug-and-Play Image Inpainting Model with Designed Pixel-Level Charge*. European Conference on Computer Vision (ECCV). <https://tencentarc.github.io/BrushNet/>
- [42] Ye, H., Zhang, J., Liu, S., Han, X., & Yang, W. (2023). *IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models*. <https://ip-adapter.github.io/>
- [43] Black Forest Labs. (2024). *FLUX.1-schnell*. <https://github.com/black-forest-labs/flux>
- [44] Li, J., et al. (2024). *Hunyuan-DiT: A Powerful Multi-Resolution Diffusion Transformer with Fine-Grained Chinese Understanding*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://dit.hunyuan.tencent.com/>
- [45] Razzhigaev, A., et al. (2023). *Kandinsky: an Open-Source Text-to-Image Synthesis Model*. <https://ai-forever.github.io/Kandinsky-3/>